

Shashwath Ks Assignment 4 Day 04

Programming Essentials For Ai

Dated : 25/04/2026

Source-W3Schools

Topic
Python Range
Python Arrays
Python While Iterators
Python Modules
Python Dates
Python Math
Python JSON
Python RegEx
Python PIP
Python Try...Except
Python String Formatting
Python None
Python User Input
Python VirtualEnv
Python Classes
Matplotlib

Python range

The built-in `range()` function returns an immutable sequence of numbers, commonly used for looping a specific number of times.

This set of numbers has its own data type called range.

Creating ranges

The `range()` function can be called with 1, 2, or 3 arguments, using this syntax:

```
range(start, stop, step)
```

Call range() With One Argument

If the range function is called with only one argument, the argument represents the stop value.

The `start` argument is optional, and if not provided, it defaults to 0.

`range(10)` returns a sequence of each number from 0 to 9. (The start argument, 0 is inclusive, and the stop argument, 10 is exclusive).

```
x = range(10) #Calling range with a single argument i.e start
```

```
x = range(3, 10) # Calling range with two arguments i.e start and end
```

```
x = range(3,10,2) # Calling range with three arguments i.e start, stop and step
```

Using Ranges

Ranges are often used in `for` loops to iterate over a sequence of numbers.

```
for i in range(10): # called a range of 0 to iterate upto 10 times ending at 9.  
    print(i) #returns the result
```

Output Block

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

Using List to Display Ranges

The range object is a data type that represents an immutable sequence of numbers, and it is not directly displayable.

Therefore, ranges are often converted to lists for display.

```
# Convert different ranges to lists:  
print(list(range(5))) # prints list of range 0 to 4  
print(list(range(1,6))) # prints list of range 1 to 5  
print(list(range(5,20,5))) # prints list of range 5 to 20 with 3 steps skips
```

Output Block

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[5, 10, 15]
```

Slicing Ranges

Like other sequences, ranges can be sliced to extract a subsequence.

```
r = range(10) # Taing a range from 0 to 9 (10 numbers)
print(r[2]) # Retriving the item in index position 2
# The following returns a Rnage object
print(r[:3]) # Slicing the list to print upto index position 3
```

Output Block

```
2
range(0, 3)
```

Membership Testing

Ranges support membership testing with the in operator.

```
# Test if the numbers 6 and 7 are present in a range:

r = range(0, 10, 2) # Defining the range of start from 0 to end 10 with 2 skips steps
print(6 in r) # Returns True if item in variable r
print(7 in r) # Returns False if item in variable r is absent
```

Output Block

```
True
False
```

Length

Ranges support the len() function to get the number of elements in the range.

```
r = range(0, 10, 2) # Defining the range of start from 0 to end 10 with 2 skips steps
print(len(r)) # Returns the length of the list
```

Output Block

```
5
```

Challenge: Range

Test your understanding of Python range by completing a small coding challenge.

Instructions Inside the editor, complete the following steps:

- Use a for loop with range(6) to print numbers 0 through 5
- Use a for loop with range() to print numbers 2 through 5

```
# Print 0 through 5
for x in range(6):
    print(x)

# Print 2 through 5
for x in range(2,6):
    print(x)
```

Note: Python does not have built-in support for Arrays, but Python Lists can be used instead.

Arrays

In python it shows how to use LISTS as ARRAYS, however, to work with arrays in Python you will have to import a library, like the NumPy library.

```
cars = ["Ford", "Volvo", "BMW"] # Create an Array containing a car name
```

What is an Array?

An **array** is a special variable that can hold more than one value at a time.

If you have a list of items (for example, a list of car names), storing them in separate variables might look like this:

```
car1 = "Ford"
car2 = "Volvo"
car3 = "BMW"
```

However, what if you want to loop through the cars and find a specific one? And what if you had not just 3 cars, but **300**?

The solution is an **array**!

An array can hold many values under a single name, and you can access those values by referring to an **index number**.

Access the Elements of an Array

You refer to an array element by using its **index number**.

Example: Get the value of the first array item

```
x = cars[0]
```

Example: Modify the value of the first array item

```
cars[0] = "Toyota"
```

The Length of an Array

Use the `len()` function to return the length of an array (the number of elements it contains).

```
x = len(cars) # Returns the length of the object
```

Looping Array Elements

You can use the for in loop to loop through all the elements of an array.

```
# Print each item in the cars array
for x in cars:
    print(x)
```

Output Block

```
Ford
Volvo
BMW
```

Adding Array Elements

You can use the `append()` method to add an element to an array.

```
# Add one more element to the cars array
cars.append("Honda") # Adds item Honda to the end of an array.
```

Removing Array Elements

You can use the `pop()` method to remove an element from the array.

```
# Delete the second element of the cars array
print(cars)
cars.pop(1) # Removes the element Volvo from the List
print(cars)
```

Output Block

```
['Ford', 'Volvo', 'BMW', 'Honda']  
['Ford', 'BMW', 'Honda']
```

```
# You can also use the remove() method to remove an element from the array.  
cars.remove("Volvo") # Removes Volvo from the list
```

Array Methods (Python Lists)

Python provides a set of built-in methods that you can use on lists (arrays):

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Adds elements of another list (or any iterable) to the end
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the first item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Python Iterators

An iterator is an object that contains a countable number of values.

An iterator is an object that can be iterated upon, meaning that you can traverse through all the values.

Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods `__iter__()` and `__next__()`.

Iterator vs Iterable

Lists, tuples, dictionaries, and sets are all iterable objects. They are iterable containers which you can get an iterator from.

All these objects have a `iter()` method which is used to get an iterator

```
# Return an iterator from a tuple, and print each value:
```

```
mytuple = ("apple", "banana", "cherry") # Create a tuple with 3 items (iterable object)
myit = iter(mytuple) # iter() method used for iterating through the iterables.
# Prints the Items in order
print(next(myit)) # apple
print(next(myit)) # banana
print(next(myit)) # cherry
```

Output Block

```
apple
banana
cherry
```

```
#Similarly using a string as an iterable
mystr = "banana"
myit = iter(mystr)
# String treats each letters as an item it can be iterated as
print(next(myit)) # Outputs: b
print(next(myit)) # Outputs: a
print(next(myit)) # Outputs: n
print(next(myit)) # Outputs: a
print(next(myit)) # Outputs: n
print(next(myit)) # Outputs: a
```

Output Block

```
b
a
n
a
n
a
```

Looping Through an Iterator

We can also use a for loop to iterate through an iterable object.

The for loop actually creates an iterator object and executes the `next()` method for each loop.

```
# Iterate the values of a tuple:

mytuple = ("apple", "banana", "cherry") # tuple as an iterable
# Using for loop as an iterator
for x in mytuple:
    print(x)
```

Output Block

```
apple
banana
cherry
```

Create an Iterator

To create an object/class as an iterator you have to implement the methods **iter()** and **next()** to your object.

As you will learn in the Python Classes/Objects chapter, all classes have a function called **init()**, which allows you to do some initializing when the object is being created.

The **iter()** method acts similar, you can do operations (initializing etc.), but must always return the iterator object itself.

The **next()** method also allows you to do operations, and must return the next item in the sequence.

```
class MyNumbers:
    def __iter__(self):
        self.a = 1          # Initialize starting value
        return self         # Return the iterator object

    def __next__(self):
        x = self.a          # Store current value
        self.a += 1         # Increment value
        return x            # Return next number

myclass = MyNumbers()      # Create object of MyNumbers
myiter = iter(myclass)     # Get iterator from object

print(next(myiter))        # Print 1
print(next(myiter))        # Print 2
print(next(myiter))        # Print 3
print(next(myiter))        # Print 4
print(next(myiter))        # Print 5
```

Output Block

```
1
2
3
4
5
```

StopIteration

The example above would continue forever if you had enough next() statements, or if it was used in a for loop.

To prevent the iteration from going on forever, we can use the **StopIteration** statement.

In the **next()** method, we can add a terminating condition to raise an error if the iteration is done a specified number of times.

Challenge: Iterators

Test your understanding of Python iterators by completing a small coding challenge.

Instructions Inside the editor, complete the following steps:

- Create a tuple called mytuple with the values "apple", "banana", "cherry"
- Create a variable myit and use iter() on the tuple
- Print the first item using next()

```
# Create a tuple
mytuple = ("apple", "banana", "cherry")
# Create an iterator
myit = iter(mytuple)
# Print the first item
print(next(myit))
```

Python Modules

A Python module is a file with a `.py` extension that acts as a container for encapsulated code, including functions, classes, and variables. By using the `import` statement, you can reference this external file to reuse its logic across different scripts, promoting a modular architecture and keeping your main application codebase organized.

```
# Save the following code in a file named "myModule.py"
person_details = {
    "name": "Abhay",
    "age": 30,
    "city": "Chennai"
}
```

```
# Import the module named mymodule, and access the person1 dictionary:
# Since this is a python notebook file all code contents are not executed
import myModule # This should be in the same directory as the myModule.py file

a = myModule.person_details["age"]
print(a)
```

Output Block

30

Re-naming a Module

You can create an `alias` when you import a module, by using the `as` keyword:

```
# Create an alias for mymodule called mx:

import myModule as mx

a = mx.person_details["age"]
```

```
print(a)
```

Built-in Modules

There are several built-in modules in Python, which you can import whenever you like.

```
# Import and use the platform module: Which returns the name of the operating system
dependent module imported.
```

```
import platform
```

```
x = platform.system()
print(x)
```

Output Block

Windows

Using the dir() Function

There is a built-in function to list all the function names (or variable names) in a module. The dir() function.

```
# List all the defined names belonging to the platform module:
```

```
import platform
```

```
x = dir(platform)
print(x)
```

Output Block

```
['AndroidVer', 'IOSVersionInfo', '_Processor', '_WIN32_CLIENT_RELEASES',
 '_WIN32_SERVER_RELEASES', '__builtins__', '__cached__', '__copyright__', '__doc__',
 '__file__', '__loader__', '__name__', '__package__', '__spec__', '__version__',
 '_comparable_version', '_default_architecture', '_follow_symlinks',
 '_get_machine_win32', '_java_getprop', '_mac_ver_xml', '_main', '_node',
 '_norm_version', '_os_release_cache', '_os_release_candidates', '_parse_args',
 '_parse_os_release', '_platform', '_platform_cache', '_sys_version',
 '_sys_version_cache', '_syscmd_file', '_syscmd_ver', '_uname_cache',
 '_unknown_as_blank', '_ver_stages', '_win32_ver', '_wmi', '_wmi_query', 'android_ver',
 'architecture', 'collections', 'freedesktop_os_release', 'functools',
 'invalidate_caches', 'ios_ver', 'itertools', 'java_ver', 'libc_ver', 'mac_ver',
 'machine', 'node', 'os', 'platform', 'processor', 'python_branch', 'python_build',
 'python_compiler', 'python_implementation', 'python_revision', 'python_version',
 'python_version_tuple', 're', 'release', 'sys', 'system', 'system_alias', 'uname',
 'uname_result', 'version', 'win32_edition', 'win32_is_iot', 'win32_ver']
```

Dates

Python lacks a native primitive date type, so it utilizes the datetime module to represent dates as complex objects containing year, month, and day attributes. This object-oriented approach allows for sophisticated operations like timedelta arithmetic and locale-specific formatting that standard strings or integers cannot perform.

```
# Import the datetime module and display the current date:

import datetime

x = datetime.datetime.now()
print(x)
```

Output Block

```
2026-04-02 23:44:09.771734
```

```
# Return the year and name of weekday:

import datetime

x = datetime.datetime.now()

print(x.year)
print(x.strftime("%A"))
```

Output Block

```
2026
Thursday
```

Creating Date Objects

To create a date, we can use the datetime() class (constructor) of the datetime module.

The datetime() class requires three parameters to create a date: year, month, day.

```
# Create a date object:
import datetime

x = datetime.datetime(2020, 5, 17) # This will create a date object representing May
17, 2020
print(x)
```

Output Block

```
2020-05-17 00:00:00
```

The strftime() Method

The datetime object has a method for formatting date objects into readable strings.

The method is called strftime(), and takes one parameter, format, to specify the format of the returned string:

```
# Display the name of the month:

import datetime
x = datetime.datetime(2018, 6, 1) # This will create a date object representing June 1, 2018
print(x.strftime("%B")) # This will print the name of the month, which is "June"
```

Output Block

```
June
```

All legal format codes

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018
%H	Hour 00-23	17
%I	Hour 00-12	05
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	08
%f	Microsecond 000000-999999	548513
%z	UTC offset	+0100
%Z	Timezone	CST
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as first day, 00-53	52
%W	Week number of year, Monday as first day, 00-53	52
%c	Local version of date and time	Mon Dec 31 17:41:00 2018
%C	Century	20
%x	Local version of date	12/31/18
%X	Local version of time	17:41:00
%%	A % character	%
%G	ISO 8601 year	2018
%u	ISO 8601 weekday (1-7)	1
%V	ISO 8601 week number (01-53)	01

Math

Python provides basic arithmetic via built-in functions like `abs()` or `round()`, but more complex operations require the `math` module. This module serves as a C-standard library wrapper, providing precise tools for trigonometric, logarithmic, and power functions that operate on floating-point numbers.

Built-in Math Functions

The `min()` and `max()` functions can be used to find the lowest or highest value in an iterable

```
x = min(5, 10, 25) # to find the minimum value among the given numbers
y = max(5, 10, 25) # to find the maximum value among the given numbers

print(x)
print(y)
```

The `math` module is a standard library extension that provides access to the underlying C-standard mathematical functions for floating-point arithmetic. By using `import math`, you gain high-precision tools for operations such as `math.sqrt()` for square roots, `math.ceil()` for rounding up, and access to mathematical constants.

```
import math
```

When you have imported the `math` module, you can start using methods and constants of the module.

The `math.sqrt()` method for example, returns the square root of a number:

```
import math

result = math.sqrt(64) # to find square root of the number

print(result)
```

Output Block

```
8.0
```

The `math.ceil()` method rounds a number upwards to its nearest integer, and the `math.floor()` method rounds a number downwards to its nearest integer, and returns the result

```
import math

x = math.ceil(1.4)
y = math.floor(1.4)

print(x) # returns 2
print(y) # returns 1
```

Output Block

```
2
1
```

```
import math
pi_value = math.pi # to get value of pi
print(pi_value)
```

JSON in Python

Python has a built-in package called json, which can be used to work with JSON data.

```
import json

data = '{ "name":"John", "age":30, "city":"New York"}'

json_string = json.loads(data)
print(json_string["age"])
```

Output Block

```
30
```

Convert from Python to JSON

If you have a Python object, you can convert it into a JSON string by using the json.dumps() method.

```
import json

# a Python object (dict):
x = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

# convert into JSON:
y = json.dumps(x)

# the result is a JSON string:
print(y)
```

Output Block

```
{"name": "John", "age": 30, "city": "New York"}
```

Python RegEx

- A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.
- RegEx can be used to check if a string contains the specified search pattern.
- Python has a built-in package called re, which can be used to work with Regular Expressions.

```
# Import the re module
import re
```

```
# Search the string to see if it starts with "The" and ends with "Spain"
import re
txt = "The rain in Spain stays mainly in the plain"
x = re.search("^The.*Spain$", txt)
```

RegEx Functions

The re module offers a set of functions that allows us to search a string for a match

Function	Description
findall	Returns a list containing all matches
search	Returns a match object if found anywhere in the string
split	Splits the string at matches and returns a list
sub	Replaces matches with a specified string

Metacharacters

Metacharacters are characters with a special meaning.

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence or escapes special characters	"\d"
.	Any character (except newline)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"planet\$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrence	"he.?o"
{}	Exactly the specified number of occurrences	"he.{2}o"
	Either or	"falls stays"
()	Capture and group	

Flags

You can add flags to the pattern when using regular expressions.

Flag	Shorthand	Description
re.ASCII	re.A	Returns only ASCII matches
re.DEBUG		Returns debug information
re.DOTALL	re.S	Makes . match all characters (including newline)
re.IGNORECASE	re.I	Case-insensitive matching
re.MULTILINE	re.M	Matches at the beginning of each line
re.NOFLAG		Specifies no flag is set
re.UNICODE	re.U	Returns Unicode matches (default in Python 3)
re.VERBOSE	re.X	Allows spaces and comments, improves readability

Special Sequences

A special sequence is a `\` followed by one of the characters in the list below, and has a special meaning:

Character	Description	Example
\A	Match at the beginning of the string	"\AThe"
\b	Match at the beginning or end of a word	r"\bain"
\B	Match not at the beginning or end of a word	r"\Bain"
\d	Match digits (0-9)	"\d"
\D	Match non-digits	"\D"
\s	Match whitespace	"\s"
\S	Match non-whitespace	"\S"
\w	Match word characters (a-z, A-Z, 0-9, _)	"\w"
\W	Match non-word characters	"\W"
\Z	Match at the end of the string	"Spain\Z"

Sets

A set is a set of characters inside a pair of square brackets [] with a special meaning.

Set	Description
[arn]	Match any of a, r, or n
[a-n]	Match any lowercase letter from a to n
[^arn]	Match any character except a, r, n
[0123]	Match any of the digits 0, 1, 2, or 3
[0-9]	Match any digit from 0 to 9
[0-5][0-9]	Match any two-digit number from 00 to 59
[a-zA-Z]	Match any letter (lowercase or uppercase)
[+]	Match the + character

The findall() Function

The findall() function returns a list containing all matches.

```
import re

txt = "The rain in Spain"
x = re.findall("ai", txt)
print(x)
```

```
#The list contains the matches in the order they are found.

# If no matches are found, an empty list is returned:

import re

txt = "The rain in Spain"
x = re.findall("Portugal", txt)
print(x)
```

The search() Function

The search() function searches the string for a match, and returns a Match object if there is a match.

If there is more than one match, only the first occurrence of the match will be returned:

```
import re

txt = "The rain in Spain"
x = re.search("\s", txt)

print("The first white-space character is located in position:", x.start())
```

Output Block

The first white-space character is located in position: 3

```
<>:4: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in
the future. Did you mean "\\s"? A raw string is also an option.
<>:4: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in
the future. Did you mean "\\s"? A raw string is also an option.
C:\Users\shash\AppData\Local\Temp\ipykernel_3768\3440849899.py:4: SyntaxWarning: "\s" is
an invalid escape sequence. Such sequences will not work in the future. Did you mean
"\s"? A raw string is also an option.
x = re.search("\s", txt)
```

```
# Make a search that returns no match:

import re

txt = "The rain in Spain"
x = re.search("Portugal", txt)
print(x)
```

Output Block

None

The split() Function

The split() function returns a list where the string has been split at each match.

```
# Split at each white-space character:
```

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

Output Block

```
['The', 'rain', 'in', 'Spain']
```

```
<>:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\s"? A raw string is also an option.
<>:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\s"? A raw string is also an option.
C:\Users\shash\AppData\Local\Temp\ipykernel_3768\3557336483.py:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\s"? A raw string is also an option.
x = re.split("\s", txt)
```

```
# Split the string only at the first occurrence:
```

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt, 1)
print(x)
```

Output Block

```
['The', 'rain in Spain']
```

```
<>:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\s"? A raw string is also an option.
<>:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\s"? A raw string is also an option.
C:\Users\shash\AppData\Local\Temp\ipykernel_3768\1604875368.py:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\s"? A raw string is also an option.
x = re.split("\s", txt, 1)
C:\Users\shash\AppData\Local\Temp\ipykernel_3768\1604875368.py:6: DeprecationWarning: 'maxsplit' is passed as positional argument
x = re.split("\s", txt, 1)
```

The sub() Function

The sub() function replaces the matches with the text of your choice.

```
# Replace every white-space character with the number 9:
```

```
import re
```

```
txt = "The rain in Spain"  
x = re.sub("\s", "9", txt)  
print(x)
```

Output Block

```
The9rain9in9Spain
```

```
<>:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in  
the future. Did you mean "\\s"? A raw string is also an option.  
<>:6: SyntaxWarning: "\s" is an invalid escape sequence. Such sequences will not work in  
the future. Did you mean "\\s"? A raw string is also an option.  
C:\Users\shash\AppData\Local\Temp\ipykernel_3768\3986763514.py:6: SyntaxWarning: "\s" is  
an invalid escape sequence. Such sequences will not work in the future. Did you mean  
"\\s"? A raw string is also an option.  
x = re.sub("\s", "9", txt)
```

The Match object has properties and methods used to retrieve information about the search, and the result:

- `.span()` returns a tuple containing the start-, and end positions of the match.
- `.string` returns the string passed into the function
- `.group()` returns the part of the string where there was a match

Print the position (start- and end-position) of the first match occurrence.

The regular expression looks for any words that starts with an upper case "S":

```
import re
```

```
txt = "The rain in Spain"  
x = re.search(r"\bS\w+", txt)  
print(x.span())
```

What is PIP?

PIP is a package manager for Python packages, or modules if you like.

What is a Package?

A package contains all the files you need for a module.

Modules are Python code libraries you can include in your project.

Check if PIP is Installed

Navigate your command line to the location of Python's script directory, and type the following:

```
PS D:\TheDeuce\File_DS> pip --version
pip 26.0.1 from C:\Users\shash\AppData\Roaming\Python\Python314\site-packages\pip
(python 3.14)
```

Download a Package

Downloading a package is very easy.

Open the command line interface and tell PIP to download the package you want.

Navigate your command line to the location of Python's script directory, and type the following.

```
PS D:\TheDeuce\File_DS> pip install camelcase
Defaulting to user installation because normal site-packages is not writeable
Collecting camelcase
  Downloading camelcase-0.2.tar.gz (1.3 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: camelcase
  Building wheel for camelcase (pyproject.toml) ... done
  Created wheel for camelcase: filename=camelcase-0.2-py3-none-any.whl size=1800
sha256=fb27fd488b37432bd4dac8485fb1fc49817dfd67096ba9339ad735c3ec09aaf1
  Stored in directory: c:\users\shash\appdata\local\pip\cache\wheels\27\21\83\3aae96a
ff2221c3e4861f8510289265a9558e8322489055add
Successfully built camelcase
Installing collected packages: camelcase
Successfully installed camelcase-0.2
```

Uninstalling Package

```
PS D:\TheDeuce\File_DS> pip uninstall camelcase
Found existing installation: camelcase 0.2
Uninstalling camelcase-0.2:
  Would remove:
    c:\users\shash\appdata\roaming\python\python314\site-packages\camelcase-0.2.dist-
info\*
    c:\users\shash\appdata\roaming\python\python314\site-packages\camelcase\*
Proceed (Y/n)? Y
Successfully uninstalled camelcase-0.2
```

Python Try Except

- The try block lets you test a block of code for errors.
- The except block lets you handle the error.
- The else block lets you execute code when there is no error.
- The finally block lets you execute code, regardless of the result of the try- and except blocks.

Exception Handling

When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

These exceptions can be handled using the try statement

```
# The try block will generate an exception, because x is not defined:

try:
    print(x)
except:
    print("An exception occurred")
```

Output Block

```
The9rain9in9Spain
```

Many Exceptions

You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error.

```
# Print one message if the try block raises a NameError and another for other errors:

try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
```

Output Block

```
The9rain9in9Spain
```

```
# In this example, the try block does not generate any error:

try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

Output Block

```
Hello
Nothing went wrong
```

```
# The finally block, if specified, will be executed regardless if the try block raises
an error or not.
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

Output Block

```
The9rain9in9Spain
The 'try except' is finished
```

```
# Try to open and write to a file that is not writable:

try:
    f = open("demofile.txt")
    try:
        f.write("Lorum Ipsum")
    except:
        print("Something went wrong when writing to the file")
    finally:
        f.close()
except:
    print("Something went wrong when opening the file")
```

Output Block

```
Something went wrong when opening the file
```

Raise an exception

As a Python developer you can choose to throw an exception if a condition occurs.

To throw (or raise) an exception, use the raise keyword.

- The raise keyword is used to raise an exception.
- You can define what kind of error to raise, and the text to print to the user.

```
# Raise an error and stop the program if x is lower than 0:

x = -1

if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

Output Block

Exception: Sorry, no numbers below zero

```
# Raise a TypeError if x is not an integer:

x = "hello"

if not type(x) is int:
    raise TypeError("Only integers are allowed")
```

Output Block

TypeError: Only integers are allowed

String Formatting

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

Before Python 3.6 we had to use the `format()` method.

F-Strings

F-string allows you to format selected parts of a string.

To specify a string as an f-string, simply put an `f` in front of the string literal, like this:

```
text_message = f"The price is 50 dollars" # creating a formatted string
print(text_message)
```

Output Block

The price is 50 dollars

In Python formatting, a modifier (placed after a colon `:`) acts as a technical directive for the interpreter on how to present a value's data. For instance, `.2f` is a type specifier that forces a float into a fixed-point representation with exactly two decimal places, handling rounding and padding automatically during the string conversion.

```
# Display the price with 2 decimals
amount = 59 # price value
formatted_text = f"The price is {amount:.2f} dollars" # format to 2 decimal places
print(formatted_text)

# Display the value 95 with 2 decimals
amount = 95 # price value
formatted_text = f"The price is {amount:.2f} dollars" # format to 2 decimal places
print(formatted_text)
```

Output Block

```
The price is 59.00 dollars
The price is 95.00 dollars
```

Perform Operations in F-Strings

- You can perform Python operations inside the placeholders.
- You can do math operations.

```
# Perform a math operation in the placeholder, and return the result:

txt = f"The price is {50 * 59} dollars"
print(txt)
```

Output Block

```
The price is 2950 dollars
```

```
# Add taxes before displaying the price:

price = 59
tax = 0.25
txt = f"The price is {price + (price * tax)} dollars"
print(txt)
```

Output Block

```
The price is 73.75 dollars
```

```
item_price = 49 # given price

result_text = f"It is very {'Expensive' if item_price > 50 else 'Cheap'}" # condition
inside f-string

print(result_text)
```

Output Block

```
It is very Cheap
```

Execute Functions in F-Strings

You can execute functions inside the placeholder.

```
# Use the string method upper() to convert a value into upper case letters:
snack = "mangoes" # fruit name
message = f"I enjoy {snack.upper()}" # convert to uppercase
print(message)
```

Output Block

```
I enjoy MANGOES
```

```
# Create a function that converts feet into meters:
def feet_to_meters(feet): # convert feet to meters
    return feet * 0.3048

altitude_text = f"The plane is flying at a {feet_to_meters(30000)} meter altitude"

print(altitude_text)
```

Output Block

```
The plane is flying at a 9144.0 meter altitude
```

More Modifiers

Python's format modifiers use a specific mini-language syntax within placeholders to control alignment, padding, and base conversion. By appending symbols like `>10` for right-alignment or `x` for hexadecimal conversion after the colon, you can programmatically standardize the visual structure of data outputs without manual string manipulation.

```
price = 59000
txt = f"The price is {price:,.} dollars" # comma used as a thousand separator
print(txt)
```

Here are some of more modifiers

Format Specifier	Description
:<	Left aligns the result (within the available space)
:>	Right aligns the result (within the available space)
:^	Center aligns the result (within the available space)
:=	Places the sign to the left most position
:+	Use a plus sign to indicate if the result is positive or negative
:-	Use a minus sign for negative values only
:	Use a space to insert an extra space before positive numbers (and a minus sign before negative numbers)
:,	Use a comma as a thousand separator
:_	Use a underscore as a thousand separator
:b	Binary format
:c	Converts the value into the corresponding Unicode character
:d	Decimal format
:e	Scientific format, with a lower case e
:E	Scientific format, with an upper case E
:f	Fix point number format
:F	Fix point number format, in uppercase format (show inf and nan as INF and NAN)
:g	General format
:G	General format (using a upper case E for scientific notations)
:o	Octal format
:x	Hex format, lower case
:X	Hex format, upper case
:n	Number format
:%	Percentage format

String `format()` Method

Prior to Python 3.6, string formatting was commonly done using the `format()` method.

While `format()` is still available and works perfectly fine, f-strings are now generally favored because they are more efficient and easier to read.

The examples that follow will show how string formatting is done using the `format()` method.

Like f-strings, the `format()` method also relies on curly braces `{}` as placeholders, though its usage and structure differ slightly.

```
price = 49
txt = "The price is {} dollars" # Adding a placeholder to display the price
print(txt.format(price))
```

Output Block

```
The price is 49 dollars
```

Multiple Values

To include additional values, simply pass more arguments into the `format()` method.

```
item_no = 5
count = 10
txt = "The price is {} dollars, item no. is {} of qty {}"
print(txt.format(price, item_no, count))
```

Output Block

```
The price is 49 dollars, item no. is 5 of qty 10
```

Index Numbers

You can specify index numbers (like `{0}` inside curly braces) to ensure each value is inserted into the intended position.

```
quantity = 3
itemno = 567
price = 49
myorder = "I want {0} pieces of item number {1} for {2:.2f} dollars."
print(myorder.format(quantity, itemno, price))
```

Output Block

```
I want 3 pieces of item number 567 for 49.00 dollars.
```

Named Indexes

You can define named placeholders (for example, `{carname}` inside curly braces). When doing so, you must pass values using the same names, like `txt.format(carname="Ford")`.

```
myorder = "I have a {carname}, it is a {model}."
print(myorder.format(carname = "Ford", model = "Mustang"))
```

Output Block

```
I have a Ford, it is a Mustang.
```

Python None

None is a unique constant in Python used to indicate the absence of any value.

It belongs to the **NoneType** data type, and it is the sole instance of that type.

NoneType

A variable can be assigned **None** to represent that it has no value or has not been initialized.

```
x = None # assigns x as none and returns None
print(x)

print(type(x)) #Returns NoneType as type
```

Output Block

```
None
<class 'NoneType'>
```

Comparing to None

When checking if a value is **None**, use the identity operators **is** or **is not**.

```
# the script returns None and No results yet
result = None
if result is None:
    print("No result yet")
else:
    print("Result is ready")
```

Output Block

```
No result yet
```

```
## the bool operators returns None as False
print(bool(None))
```

Output Block

```
False
```

Functions returning None

Functions that do not explicitly return a value return None by default.

```
# A function without a return statement returns None:
def myfunc():
    x = 5

x = myfunc()
print(x)
```

Output Block

None

User Input

Python supports taking input from users, allowing programs to interact dynamically.

You can prompt the user for input, and once a value is entered, it can be displayed or used within the program. For example, you can ask for a name and then print it on the screen.

```
print("Enter your name:")
name = input() # input() functions take input during the runtime
print(f"Hello {name}")
```

Output Block

Enter your name:
Hello Shash

Using Prompt

In the previous example, the user entered their input on a separate line. The `input()` function in Python includes a prompt parameter, allowing you to display a message on the same line where the user provides their input.

```
# Add a message in front of the user input:
name = input("Enter your name:")
print(f"Hello {name}")
```

Output Block

Hello Shash

Input Number

User input is always received as a string. Even if a numeric value is entered, Python will interpret it as text.

To work with numerical values, you can convert the input using functions like `float()`.

```
import math
x = input("Enter a number:")

#find the square root of the number:
y = math.sqrt(float(x))

print(f"The square root of {x} is {y}")
```

Output Block

```
The square root of 55 is 7.416198487095663
```

Validate Input

It is important to check user input before using it. In the previous example, an error would occur if the user enters a non-numeric value.

To prevent this, you can validate the input. If it is not a number, display a message such as "Wrong input, please try again" and prompt the user to enter a valid value again.

```
y = True
while y == True:
    x = input("Enter a number:")
    try:
        x = float(x);
        y = False
    except:
        print("Wrong input, please try again.")

print("Thank you!")
```

What is a Virtual Environment?

A virtual environment in Python is a self-contained workspace on your system where you can develop and run projects independently.

It helps you handle project-specific dependencies without affecting other projects or the main Python installation.

You can think of a virtual environment as an isolated container for each project. Each container:

- Uses its own Python interpreter
- Maintains its own installed packages
- Operates independently from other environments
- Can use different versions of the same package

Using virtual environments is important because:

- It avoids conflicts between package versions across projects

- Makes projects easier to share and reproduce
- Keeps the global Python setup clean
- Enables testing with different Python versions

Creating a Virtual Environment

Python provides a built-in module called `venv` to create virtual environments.

To set one up, open your command prompt, go to your project directory, and run the appropriate command:

in the terminal creating a virtual environment with the name venv

```
py -3.11 -m venv venv
```

Python OOP

What is OOP?

OOP stands for **Object-Oriented Programming**.

Python is an object-oriented language, allowing you to structure your code using **classes** and **objects** for better organization and reusability.

Advantages of OOP

- Provides a clear structure to programs
- Makes code easier to maintain, reuse, and debug
- Helps keep your code **DRY (Don't Repeat Yourself)**
- Allows you to build reusable applications with less code

Tip

The **DRY principle** means you should avoid writing the same code more than once. Move repeated code into functions or classes and reuse it.

What are Classes and Objects?

Classes and **objects** are the two core concepts in object-oriented programming.

A **class** defines what an object should look like, and an **object** is an instance created from that class.

Examples

Class	Objects
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

Key Point

When you create an object from a class, it inherits all the **variables** and **functions** defined inside that class.

Python Classes/Objects

Python is an object oriented programming language.

Almost everything in Python is an object, with its properties and methods.

A Class is like an object constructor, or a "blueprint" for creating objects.

Create a Class

To create a class, use the keyword class.

```
# Create a class named MyClass, with a property named x
class MyClass:
    x = 5 # x is assigned 5
```

```
# Create an object named p1, and print the value of x:
p1 = MyClass()
print(p1.x) # Retrives the value of x and prints it
```

Output Block

5

Delete Objects

You can delete objects by using the del keyword.

```
del p1 # Deletes the object and frees the memory.
```

Multiple Objects

You can create multiple objects from the same class.

```
# Create three objects from the MyClass class:

p1 = MyClass() # Will refer to the MyClass defined earlier
p2 = MyClass() # p2 is an another insatnce of the class
```

```
p3 = MyClass() # similar with p3
# all the 3 will return the same values of the variables
print(p1.x)
print(p2.x)
print(p3.x)
```

Output Block

```
5
5
5
```

The pass Statement

class definitions cannot be empty, but if you for some reason have a class definition with no content, put in the `pass` statement to avoid getting an error.

```
class Person:
    pass
```

The `\_\_init\_\_` Method

All classes have a built-in method called `__init__()`, which is always executed when the class is being initiated.

The `__init__()` method is used to assign values to object properties, or to perform operations that are necessary when the object is being created.

```
# Create a class named Person, use the __init__() method to assign values for name and age:
```

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil", 36) # Create an object p1 and call the Class with the parameters
```

```
print(p1.name) # Returns the Name from the class
print(p1.age) # Returns the Age item from the class
```

Output Block

```
Emil
36
```

Why Use `\_\_init\_\_`?

Without the `__init__()` method, you would need to set properties manually for each object.

```
# Create a class without __init__():
class Person:
    pass # Class (Empty) with no functions and variables

p1 = Person() # Create an Object p1
p1.name = "Tobias" # Assign a variable associated to that object
p1.age = 25 # Assign the age associated to that class

print(p1.name) # Return the name related to that object from the class
print(p1.age) # Return the age related to that object from the class
```

Output Block

```
Tobias
25
```

```
# With __init__(), you can set initial values when creating the object.

class Person:
    # Creating a init function to assign the initial values for the class objects
    def __init__(self, name, age):
        self.name = name
        self.age = age
# Creating the Object p1 and passing the values in arguments
p1 = Person("Linus", 28)
# Printing the values of the variables
print(p1.name)
print(p1.age)
```

Default Values in `\_\_init\_\_()`

You can also set default values for parameters in the `__init__()` method.

```
# Set a default value for the age parameter:

class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age

p1 = Person("Emil")
p2 = Person("Tobias", 25)

print(p1.name, p1.age)
print(p2.name, p2.age)
```

Output Block

```
Emil 18
Tobias 25
```

Multiple Parameters

The `__init__()` method can have as many parameters as you need

```
# Create a person class with multiple parameters

class Person:
    def __init__(self, name, age, city, country):
        self.name = name
        self.age = age
        self.city = city
        self.country = country

# Creating an object for the instance of the class
p1 = Person("Linus", 20, "Torvalds", "Norway")
print(f"Name : {p1.name}\nAge: {p1.age}\nCity: {p1.city}\nCountry: {p1.country}")
```

Output Block

```
Name : Linus
Age: 20
City: Torvalds
Country: Norway
```

The self Parameter

The `self` parameter is a reference to the current instance of the class.

It is used to access properties and methods that belong to the class.

```
class Person:
    def __init__(self, name, age):
        self.name = name # Assign name to instance variable
        self.age = age # Assign age to instance variable

    def greet(self):
        print("Hello, my name is " + self.name) # Access name using self

p1 = Person("Emil", 25) # Create object with name and age
p1.greet() # Call method using the object
```

Output Block

```
Hello, my name is Emil
```

Why Use self?

Without `self`, Python would not know which object's properties you want to access

```
class Person:
    def __init__(self, name):
```

```

        self.name = name # Store name in instance variable

    def printname(self):
        print(self.name) # Print the stored name

p1 = Person("Tobias") # Create first object
p2 = Person("Linus") # Create second object

p1.printname()
p2.printname()

```

Output Block

```

Tobias
Linus

```

`self` Does Not Have to Be Named "self"

It does not have to be named self, you can call it whatever you like, but it has to be the first parameter of any method in the class.

```

class Person:
    def __init__(myobject, name, age):
        myobject.name = name # assign name to object
        myobject.age = age # assign age to object

    def greet(abc):
        print("Hello, my name is " + abc.name) # access name using object reference

p1 = Person("Emil", 36) # create Person object
p1.greet() # call greet method

```

Output Block

```

Hello, my name is Emil

```

Accessing Properties with self

You can access any property of the class using self.

```

class Car:
    def __init__(self, brand, model, year):
        self.brand = brand # store brand in object
        self.model = model # store model in object
        self.year = year # store year in object

    def display_info(self):
        print(f"{self.year} {self.brand} {self.model}") # print car details

car1 = Car("Toyota", "Corolla", 2020) # create car object
car1.display_info() # display car info

```

2020 Toyota Corolla

Calling Methods with self

You can also call other methods within the class using self.

```
# To demonstrate calling an function within a class
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello : {self.name}"

    def welcome(self):
        message = self.greet() # Calling the function within the class
        print(f"{message} : Welcome to our website!")

# Creating a class object named p1
p1 = Person("Tobias") # Passing argument with the name
p1.welcome() # Calling the function inside the variable to greet the name passed
initially by the user when creating the object
```

Output Block

Hello : Tobias : Welcome to our website!

Class Properties

Properties are variables that belong to a class. They store data for each object created from the class.

```
#Create a class with properties
class Person:
    def __init__(self,name,age):#constructor to initialize values
        self.name=name #store name in instance
        self.age=age #store age in instance

p1=Person("Emil",36)#create object of Person

print(p1.name)#print name
print(p1.age)#print age
```

Output Block

Emil
36

Access Properties

You can access object properties using **dot** notation.

```
#Access object properties using dot notation
class Car:
    def __init__(self, brand, model): #initialize brand and model
        self.brand=brand #assign brand to instance
        self.model=model #assign model to instance

car1=Car("Toyota", "Corolla")#create Car object

print(car1.brand)#print brand
print(car1.model)#print model
```

Output Block

```
Toyota
Corolla
```

Modify Properties

You can modify the value of properties on objects.

```
#Create Person class
class Person:
    def __init__(self, name, age): #initialize name and age
        self.name=name #store name
        self.age=age #store age

p1=Person("Tobias", 25) #create object

print(p1.age) #print age
p1.age=26 #change age property
print(p1.age)#print updated age
```

Output Block

```
25
26
```

Delete Properties

You can delete properties from objects using the del keyword.

```
# Delete the age property:

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Linus", 30)
```



```
del p1.age

print(p1.name) # This works
# print(p1.age) # This would cause an error
```

Output Block

Linus

Class Properties vs Object Properties

Properties defined inside `__init__()` belong to each object (instance properties).

Properties defined outside methods belong to the class itself (class properties) and are shared by all objects.

```
#Class property vs instance property

class Person:
    species="Human" # Class property

    def __init__(self,name):
        self.name=name # Instance property

p1=Person("Emil")
p2=Person("Tobias")

print(p1.name)
print(p2.name)
print(p1.species)
print(p2.species)
```

Output Block

Emil
Tobias
Human
Human

Modifying Class Properties

When you modify a class property, it affects all objects.

```
#Change class property

class Person:
    lastname="" # Class property

    def __init__(self,name):
        self.name=name # Instance property

p1=Person("Linus") # Passing the name in the object parameter
p2=Person("Emil")
```

```
Person.lastname="Torvalds" # Change class property

print(p1.lastname)
print(p2.lastname)
```

Output Block

```
Torvalds
Torvalds
```

Add New Properties

You can add new properties to existing objects.

```
#Add attributes dynamically to an object

class Person:
    def __init__(self,name):
        self.name=name # Instance property

p1=Person("Tobias") # Create object

p1.age=25 # Add age attribute
p1.city="Oslo" # Add city attribute

print(p1.name)
print(p1.age)
print(p1.city)
```

Output Block

```
Tobias
25
Oslo
```

Python Class Methods

Methods are functions that belong to a class. They define the behavior of objects created from the class.

```
#Class method example

class Person:
    def __init__(self,name):
        self.name=name # Instance property

    def greet(self):
        print("Hello,my name is "+self.name) # Method using instance property

p1=Person("Emil") # Create object
p1.greet() # Call method
```

Output Block

```
Hello,my name is Emil
```

Methods with Parameters

Methods can accept parameters just like regular functions.

```
#Method with parameters

class Calculator:
    def add(self,a,b):
        return a+b # return sum

    def multiply(self,a,b):
        return a*b # return product

calc=Calculator() # create object

print(calc.add(5,3)) # call add method
print(calc.multiply(4,7)) # call multiply method
```

Output Block

```
8
28
```

Methods Accessing Properties

Methods can access and modify object properties using `self`.

```
#Method accessing object properties

class Person:
    def __init__(self,name,age):
        self.name=name # store name
        self.age=age # store age

    def get_info(self):
        return f"{self.name} is {self.age} years old" # use instance properties

p1=Person("Tobias",28) # create object

print(p1.get_info()) # call method
```

Output Block

```
Tobias is 28 years old
```

Methods Modifying Properties

Methods can modify the properties of an object.

```
#Method that modifies object property

class Person:
    def __init__(self, name, age):
        self.name=name # store name
        self.age=age # store age

    def celebrate_birthday(self):
        self.age+=1 # increase age
        print(f"Happy birthday! You are now {self.age}") # print updated age

p1=Person("Linus",25) # create object
p1.celebrate_birthday() # first birthday call
p1.celebrate_birthday() # second birthday call
```

Output Block

```
Happy birthday! You are now 26
Happy birthday! You are now 27
```

The __str__() Method

The **str()** method is a special method that controls what is returned when the object is printed.

```
# Without the __str__() method:

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
# Create an object named p1 passing args with name and age
p1 = Person("Emil", 36)
print(p1)
```

Output Block

```
<__main__.Person object at 0x000001999A9F1940>
```

```
# With the __str__() method:

class Person:
    def __init__(self, name, age):
        self.name = name # name variable
        self.age = age # age variable

    def __str__(self):
        return f"{self.name} ({self.age})"

# Similarly calling the class by creating the object
p1 = Person("Tobias", 36)
print(p1)
```

Multiple Methods

A class can have multiple methods that work together.

```
#Class with multiple methods

class Playlist:
    def __init__(self,name):
        self.name=name # store playlist name
        self.songs=[] # list to store songs

    def add_song(self,song):
        self.songs.append(song) # add song to list
        print(f"Added: {song}") # confirmation message

    def remove_song(self,song):
        if song in self.songs: # check if song exists
            self.songs.remove(song) # remove song
            print(f"Removed: {song}") # confirmation message

    def show_songs(self):
        print(f"Playlist '{self.name}':") # show playlist name
        for song in self.songs: # loop through songs
            print(f"- {song}") # print each song

my_playlist=Playlist("Favorites") # create playlist object
my_playlist.add_song("Bohemian Rhapsody") # add song
my_playlist.add_song("Stairway to Heaven") # add song
my_playlist.show_songs() # display songs
```

Output Block

```
Added: Bohemian Rhapsody
Added: Stairway to Heaven
Playlist 'Favorites':
- Bohemian Rhapsody
- Stairway to Heaven
```

Delete Methods

You can delete methods from a class using the del keyword.

```
#Delete a method from a class

class Person:
    def __init__(self,name):
        self.name=name # store name

    def greet(self):
        print("Hello!") # greet method

p1=Person("Emil") # create object

del Person.greet # delete method from class
```

```
# p1.greet() # this will cause an error because method is deleted
```

Python Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

- **Parent class:** The class being inherited from, also called the **base class**.
- **Child class:** The class that inherits from another class, also called the **derived class**.

Create a Parent Class

Any class can be a parent class, so the syntax is the same as creating any other class.

```
# Create a class named Person, with firstname and lastname properties, and a printname method.
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

#Use the Person class to create an object, and then execute the printname method:

x = Person("Sha", "Dow")
x.printname()
```

Output Block

Sha Dow

Create a Child Class

To create a class that inherits the functionality from another class, send the parent class as a parameter when creating the child class.

```
# Create a class named Student, which will inherit the properties and methods from the Person class:
class Student(Person):
    pass # Empty class
```

```
# Use the Student class to create an object, and then execute the printname method:
x = Student("Mike", "Olsen")
x.printname()
```

Output Block

Add the `\_\_init\_\_()` Function

So far, we have created a child class that inherits the properties and methods from its parent.

Now, we want to add the `__init__()` function to the child class (instead of using the `pass` keyword).

Note

The `__init__()` function is called **automatically** every time the class is used to create a new object.

```
# Parent class definition: Person
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname # Initialize first name
        self.lastname = lname # Initialize last name

    def printname(self):
        print(self.firstname, self.lastname) # Print full name

# Child class definition: Student, which inherits from Person
class Student(Person):
    def __init__(self, fname, lname, year):
        super().__init__(fname, lname) # Call the parent constructor to initialize
name
        self.graduationyear = year # Add new property for graduation year

    def welcome(self):
        print(f"Welcome {self.firstname} {self.lastname} to the class of
{self.graduationyear}") # Custom method for Student

# Create Person object
person1 = Person("John", "Doe")
person1.printname() # Call method from Person class

# Create Student object
student1 = Student("Mike", "Olsen", 2022)
student1.printname() # Call inherited method from Person class
student1.welcome() # Call method from Student class
```

Output Block

```
John Doe
Mike Olsen
Welcome Mike Olsen to the class of 2022
```

Python Polymorphism

- **Definition:** The word "polymorphism" means "many forms". In programming, it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

Function Polymorphism

- A function like `len()` can be used on different types of objects such as strings, tuples, and dictionaries, returning different results based on the object type.

Class Polymorphism

- Polymorphism can be applied to class methods, allowing multiple classes to have methods with the same name but different behaviors.

Inheritance Class Polymorphism

- Polymorphism can also be used with child classes inheriting from a parent class. Child classes can override inherited methods, allowing different behaviors while still following the same method name.

Additional Details to Help Understand Python Polymorphism

Key Concepts in Polymorphism

1 Method Overloading vs. Method Overriding

Method Overloading: In some programming languages (like Java), you can have multiple methods with the same name but different parameters. Python doesn't support method overloading in the traditional sense, but you can achieve similar behavior by using default arguments or variable-length argument lists (`*args` or `**kwargs`).

- 2 **Method Overriding:** This happens when a child class defines a method with the same name as one in the parent class. The child class method **overrides** the parent class method, which means the child's version of the method will be called, not the parent's.

Benefits of Polymorphism

- **Flexibility:** Polymorphism allows you to write code that can work with objects of different classes in a generic way. This reduces the complexity of the code and makes it more flexible and reusable.
- **Maintainability:** By using polymorphism, you can modify or extend your code without changing the core structure, leading to cleaner and more maintainable code.
- **Simplifies Code:** You don't need to write multiple `if-else` or `switch` statements to handle different object types. Polymorphism lets you call the same method on different objects, regardless of their class.

Polymorphism in Practice

- **Real-world analogy:** Consider a `Shape` class with methods like `area()` and `perimeter()`. Different subclasses like `Circle`, `Rectangle`, and `Triangle` will all have their own implementations of these methods, but the method name stays the same. So, when you call `shape.area()`, Python knows the exact behavior based on the object type (circle, rectangle, etc.), even though you are calling the same method name.

- **Dynamic Dispatch:** Python's polymorphism works through dynamic dispatch, meaning the method call is resolved at runtime. This is one of the reasons why polymorphism in Python is so flexible—methods are bound to objects at runtime, not at compile-time.

Practical Example: Using Polymorphism in Collections

Polymorphism is particularly useful when you want to deal with a collection of objects, where the objects can belong to different classes but implement the same methods. For example, you could store a variety of different vehicles in a list, and call the same `move()` method on each, even though `move()` behaves differently for each class (Car, Boat, Plane).

```
# Polymorphism in Python

# Parent class Vehicle
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand # Brand of the vehicle
        self.model = model # Model of the vehicle

    def move(self):
        print("Move!") # Default move method for Vehicle

# Child class Car, inherits from Vehicle
class Car(Vehicle):
    def move(self):
        print("Drive!") # Overridden move method for Car

# Child class Boat, inherits from Vehicle
class Boat(Vehicle):
    def move(self):
        print("Sail!") # Overridden move method for Boat

# Child class Plane, inherits from Vehicle
class Plane(Vehicle):
    def move(self):
        print("Fly!") # Overridden move method for Plane

# Create objects of each class
car1 = Car("Ford", "Mustang")
boat1 = Boat("Ibiza", "Touring 20")
plane1 = Plane("Boeing", "747")

# List of objects to iterate through
vehicles = [car1, boat1, plane1]

# Call move() method for each object in the list
for vehicle in vehicles:
    print(f"{vehicle.brand} {vehicle.model} - ", end="")
    vehicle.move() # Polymorphism in action
```

Output Block

Ford Mustang - Drive!
Ibiza Touring 20 - Sail!
Boeing 747 - Fly!

Python Encapsulation

Encapsulation is about protecting data inside a class. It involves keeping data (properties) and methods together while controlling how the data can be accessed from outside the class. This helps in preventing accidental changes to data and hides internal details of the class's workings.

Private Properties

In Python, properties can be made **private** by using a double underscore `__` prefix.

- **Example:** You can create a private class property such as `__age` to restrict access from outside the class.

Accessing Private Property Value

To access private properties, you can create a **getter method**.

- **Example:** Use a method like `get_age()` to access the value of a private property like `__age`.

Modifying Private Property Value

To modify a private property, you can create a **setter method**. The setter method can also validate the value before modifying the property.

- **Example:** Use `set_age()` to change the value of a private property like `__age`, ensuring it's a valid positive number.

Why Use Encapsulation?

- **Data Protection:** Prevents accidental modification of data.
- **Validation:** Allows you to validate data before it is set.
- **Flexibility:** Internal implementation can change without affecting external code.
- **Control:** You maintain control over how data is accessed and modified.

Protected Properties

Python uses a **single underscore** `_` for **protected** properties. It's a convention indicating that the property should not be accessed directly, but Python doesn't enforce this restriction.

- **Example:** A property like `_salary` is intended for internal use, but can still be accessed outside the class.

Private Methods

You can also make methods **private** using the double underscore `__` prefix.

- **Example:** Create a private method like `__validate()` that can only be used within the class itself.

Name Mangling

Python uses **name mangling** for private properties and methods. This means that private attributes (with double underscores) are internally renamed by adding `_ClassName` in front of the name.

- **Example:** A private property like `__age` becomes `_Person__age`. While you can technically access it, it's not recommended because it defeats the purpose of encapsulation.

Encapsulation helps keep your code secure, organized, and easy to maintain by controlling how data is accessed and manipulated.

```
# Encapsulation in Python

# Class with Encapsulation to protect data (private properties)
class Person:
    def __init__(self, name, age):
        self.name = name # Public property
        self.__age = age # Private property (Encapsulation)

    # Getter method to access private property
    def get_age(self):
        return self.__age

    # Setter method to modify private property with validation
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Age must be positive")

# Create an instance of Person
p1 = Person("Tobias", 25)

# Access public property
print(p1.name)

# Try accessing private property directly (will cause an error)
# print(p1.__age) # Uncommenting this will cause an AttributeError

# Correct way to access private property using getter method
print(p1.get_age())

# Modify private property using setter method
p1.set_age(26)
print(p1.get_age()) # After modification
```

```

# Try setting invalid age (negative number)
p1.set_age(-5)

# Class with Encapsulation for student grade validation
class Student:
    def __init__(self, name):
        self.name = name
        self.__grade = 0 # Private property for grade

    # Getter for grade
    def get_grade(self):
        return self.__grade

    # Setter for grade with validation
    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
        else:
            print("Grade must be between 0 and 100")

    # Method to check student's status based on grade
    def get_status(self):
        if self.__grade >= 60:
            return "Passed"
        else:
            return "Failed"

# Create a student object and set grade with validation
student1 = Student("Emil")
student1.set_grade(85) # Valid grade
print(student1.get_grade())
print(student1.get_status()) # Check student's status

# Try setting an invalid grade
student1.set_grade(150) # Invalid grade, will print an error message

# Class with a private method for calculation
class Calculator:
    def __init__(self):
        self.result = 0

    # Private method (not meant to be accessed outside the class)
    def __validate(self, num):
        if not isinstance(num, (int, float)):
            return False
        return True

    # Public method to add number
    def add(self, num):
        if self.__validate(num):
            self.result += num
        else:
            print("Invalid number")

# Create Calculator object
calc = Calculator()
calc.add(10) # Valid number

```

```
calc.add(5) # Valid number
print(calc.result) # Output the result

# Try calling the private method directly (will cause an error)
# calc.__validate(5) # Uncommenting this will cause an AttributeError
```

Output Block

```
Tobias
25
26
Age must be positive
85
Passed
Grade must be between 0 and 100
15
```

Python Inner Classes

Inner classes are classes defined inside another class. The inner class can access the properties and methods of the outer class. They help in organizing code and grouping related classes that are only used within the context of the outer class.

Creating an Inner Class

You can create an inner class by simply defining a class inside another class.

- **Example:**

Accessing the Inner Class from the Outside

To access the inner class, you need to create an instance of the outer class first, then create an object of the inner class.

- **Example:**

Accessing the Outer Class from the Inner Class

Inner classes do not automatically have access to the outer class instance. You can pass the outer class instance as a parameter to the inner class to allow access.

- **Example:**

Practical Example: Using Inner Classes for Helpers

Inner classes are ideal for representing helper objects that are only used within the outer class.

- **Example:**

Multiple Inner Classes

A class can contain multiple inner classes. Each inner class can perform different tasks while still being encapsulated within the outer class.

- **Example:**

Inner classes provide a way to organize your code and encapsulate related functionality, especially when the inner class is not needed outside the scope of the outer class.

```
# Python Inner Classes

# Outer class with an inner class
class Outer:
    def __init__(self):
        self.name = "Outer Class" # Outer class property

    # Inner class defined inside the Outer class
    class Inner:
        def __init__(self):
            self.name = "Inner Class" # Inner class property

        def display(self):
            print("This is the inner class") # Display message from the inner class

# Creating an object of Outer class
outer = Outer()
print(outer.name) # Access outer class property

# Accessing Inner class from the outside
inner = outer.Inner() # Create an instance of Inner class
inner.display() # Call method of Inner class

# Accessing Outer class from Inner class
class Outer:
    def __init__(self):
        self.name = "Emil"

    class Inner:
        def __init__(self, outer):
            self.outer = outer # Passing the outer class instance

        def display(self):
            print(f"Outer class name: {self.outer.name}") # Accessing outer class
property

# Creating an object of Outer class
outer = Outer()

# Creating an object of Inner class and passing the outer class instance
inner = outer.Inner(outer)
inner.display() # Access property of Outer class inside Inner class
```

```

# Practical example of using an inner class for encapsulation (Car and Engine)
class Car:
    def __init__(self, brand, model):
        self.brand = brand # Car's brand
        self.model = model # Car's model
        self.engine = self.Engine() # Initialize the inner Engine class

    # Inner class for Engine
    class Engine:
        def __init__(self):
            self.status = "Off" # Engine status

        def start(self):
            self.status = "Running" # Start engine
            print("Engine started")

        def stop(self):
            self.status = "Off" # Stop engine
            print("Engine stopped")

    # Method to drive the car
    def drive(self):
        if self.engine.status == "Running":
            print(f"Driving the {self.brand} {self.model}")
        else:
            print("Start the engine first!")

# Create a Car object and drive it
car = Car("Toyota", "Corolla")
car.drive() # Attempt to drive without starting the engine
car.engine.start() # Start the engine using inner class method
car.drive() # Now the car can be driven

# Example of multiple inner classes
class Computer:
    def __init__(self):
        self.cpu = self.CPU() # Initialize the inner CPU class
        self.ram = self.RAM() # Initialize the inner RAM class

    # Inner class CPU
    class CPU:
        def process(self):
            print("Processing data...") # CPU processing data

    # Inner class RAM
    class RAM:
        def store(self):
            print("Storing data...") # RAM storing data

# Create a Computer object and use the inner classes
computer = Computer()
computer.cpu.process() # Call method of inner CPU class
computer.ram.store() # Call method of inner RAM class

```

Output Block

```
Outer Class
This is the inner class
Outer class name: Emil
Start the engine first!
Engine started
Driving the Toyota Corolla
Processing data...
Storing data...
```

Python Matplotlib

1. Import Required Libraries

Import seaborn, matplotlib.pyplot, pandas, and numpy.

```
# Version 1 : With White Grid
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# Set style for seaborn
sns.set(style="whitegrid")
%matplotlib inline
```

```
# Version 2 : With Dark grid
# Imported modules with aliasing
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Set the color and styles for the Seaborn
sns.set(style='darkgrid') # used darkgrid for the visualizations
%matplotlib inline
```

2. Load the Seaborn Titanic Dataset

Load the Titanic dataset using seaborn's `load_dataset` function.

```
titanic = sns.load_dataset("titanic") # load the titanic dataset from the seaborn
library
titanic.head() # Display the first 5 rows
```

Output Block


```
survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third
```

```
who adult_male deck embark_town alive alone
0 man True NaN Southampton no False
1 woman False C Cherbourg yes False
2 woman False NaN Southampton yes True
3 woman False C Southampton yes False
4 man True NaN Southampton no True
```

3. Basic Data Exploration

Display the first few rows and summary statistics of the dataset.

```
titanic_save = titanic.to_csv("titanic_dataset.csv", index=False)
print("Saved Titanic dataset to the local drive.")
```

Output Block

```
Saved Titanic dataset to the local drive.
```

```
# Display first 5 rows
titanic.head()
```

Output Block

```
survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third
```

```
who adult_male deck embark_town alive alone
0 man True NaN Southampton no False
1 woman False C Cherbourg yes False
2 woman False NaN Southampton yes True
3 woman False C Southampton yes False
4 man True NaN Southampton no True
```

```
# Display summary statistics
# Display the summary statistics for the numerical columns only
titanic.describe()
```

Output Block

```
survived pclass age sibsp parch fare
count 891.000000 891.000000 714.000000 891.000000 891.000000 891.000000
mean 0.383838 2.308642 29.699118 0.523008 0.381594 32.204208
std 0.486592 0.836071 14.526497 1.102743 0.806057 49.693429
min 0.000000 1.000000 0.420000 0.000000 0.000000 0.000000
25% 0.000000 2.000000 20.125000 0.000000 0.000000 7.910400
50% 0.000000 3.000000 28.000000 0.000000 0.000000 14.454200
75% 1.000000 3.000000 38.000000 1.000000 0.000000 31.000000
max 1.000000 3.000000 80.000000 8.000000 6.000000 512.329200
```

```
# Describe the statistics for all the columns numerical and categorical
titanic.describe(include="all")
```

Output Block

```
survived pclass sex age sibsp parch \
count 891.000000 891.000000 891 714.000000 891.000000 891.000000
unique NaN NaN 2 NaN NaN NaN
top NaN NaN male NaN NaN NaN
freq NaN NaN 577 NaN NaN NaN
mean 0.383838 2.308642 NaN 29.699118 0.523008 0.381594
std 0.486592 0.836071 NaN 14.526497 1.102743 0.806057
min 0.000000 1.000000 NaN 0.420000 0.000000 0.000000
25% 0.000000 2.000000 NaN 20.125000 0.000000 0.000000
50% 0.000000 3.000000 NaN 28.000000 0.000000 0.000000
75% 1.000000 3.000000 NaN 38.000000 1.000000 0.000000
max 1.000000 3.000000 NaN 80.000000 8.000000 6.000000
```

```
fare embarked class who adult_male deck embark_town alive \
count 891.000000 889 891 891 891 203 889 891
unique NaN 3 3 3 2 7 3 2
top NaN S Third man True C Southampton no
freq NaN 644 491 537 537 59 644 549
mean 32.204208 NaN NaN NaN NaN NaN NaN NaN
std 49.693429 NaN NaN NaN NaN NaN NaN NaN
min 0.000000 NaN NaN NaN NaN NaN NaN NaN
25% 7.910400 NaN NaN NaN NaN NaN NaN NaN
50% 14.454200 NaN NaN NaN NaN NaN NaN NaN
75% 31.000000 NaN NaN NaN NaN NaN NaN NaN
max 512.329200 NaN NaN NaN NaN NaN NaN NaN
```

```
alone
count 891
unique 2
top True
freq 537
mean NaN
std NaN
min NaN
25% NaN
50% NaN
75% NaN
max NaN
```

```
# Describe the columns for the number specific only explicitly define
titanic.describe(include=["number"])
```

Output Block

```
survived pclass age sibsp parch fare
count 891.000000 891.000000 714.000000 891.000000 891.000000 891.000000
mean 0.383838 2.308642 29.699118 0.523008 0.381594 32.204208
std 0.486592 0.836071 14.526497 1.102743 0.806057 49.693429
min 0.000000 1.000000 0.420000 0.000000 0.000000 0.000000
25% 0.000000 2.000000 20.125000 0.000000 0.000000 7.910400
50% 0.000000 3.000000 28.000000 0.000000 0.000000 14.454200
75% 1.000000 3.000000 38.000000 1.000000 0.000000 31.000000
max 1.000000 3.000000 80.000000 8.000000 6.000000 512.329200
```

```
# Specific dtypes like int64 and float32
titanic.describe(include=["int64", "float32"])
```

Output Block

```
survived pclass sibsp parch
count 891.000000 891.000000 891.000000 891.000000
mean 0.383838 2.308642 0.523008 0.381594
std 0.486592 0.836071 1.102743 0.806057
min 0.000000 1.000000 0.000000 0.000000
25% 0.000000 2.000000 0.000000 0.000000
50% 0.000000 3.000000 0.000000 0.000000
75% 1.000000 3.000000 1.000000 0.000000
max 1.000000 3.000000 8.000000 6.000000
```

```
# Exclude certain types
titanic.describe(exclude=["object"])
```

Output Block

```

survived pclass age sibsp parch \
count 891.000000 891.000000 714.000000 891.000000 891.000000
unique NaN NaN NaN NaN NaN
top NaN NaN NaN NaN NaN
freq NaN NaN NaN NaN NaN
mean 0.383838 2.308642 29.699118 0.523008 0.381594
std 0.486592 0.836071 14.526497 1.102743 0.806057
min 0.000000 1.000000 0.420000 0.000000 0.000000
25% 0.000000 2.000000 20.125000 0.000000 0.000000
50% 0.000000 3.000000 28.000000 0.000000 0.000000
75% 1.000000 3.000000 38.000000 1.000000 0.000000
max 1.000000 3.000000 80.000000 8.000000 6.000000

```

```

fare class adult_male deck alone
count 891.000000 891 891 203 891
unique NaN 3 2 7 2
top NaN Third True C True
freq NaN 491 537 59 537
mean 32.204208 NaN NaN NaN NaN
std 49.693429 NaN NaN NaN NaN
min 0.000000 NaN NaN NaN NaN
25% 7.910400 NaN NaN NaN NaN
50% 14.454200 NaN NaN NaN NaN
75% 31.000000 NaN NaN NaN NaN
max 512.329200 NaN NaN NaN NaN

```

```

# Customize the percentiles
titanic.describe(percentiles=[0.1, 0.5, 0.9])

```

Output Block

```

survived pclass age sibsp parch fare
count 891.000000 891.000000 714.000000 891.000000 891.000000 891.000000
mean 0.383838 2.308642 29.699118 0.523008 0.381594 32.204208
std 0.486592 0.836071 14.526497 1.102743 0.806057 49.693429
min 0.000000 1.000000 0.420000 0.000000 0.000000 0.000000
10% 0.000000 1.000000 14.000000 0.000000 0.000000 7.550000
50% 0.000000 3.000000 28.000000 0.000000 0.000000 14.454200
90% 1.000000 3.000000 50.000000 1.000000 2.000000 77.958300
max 1.000000 3.000000 80.000000 8.000000 6.000000 512.329200

```

```

# Describe a single column
titanic["age"].describe()

```

Output Block

```
count 714.000000
mean 29.699118
std 14.526497
min 0.420000
25% 20.125000
50% 28.000000
75% 38.000000
max 80.000000
Name: age, dtype: float64
```

4. Bar Plot: Survival Count

Create a bar plot showing the count of survivors and non-survivors.

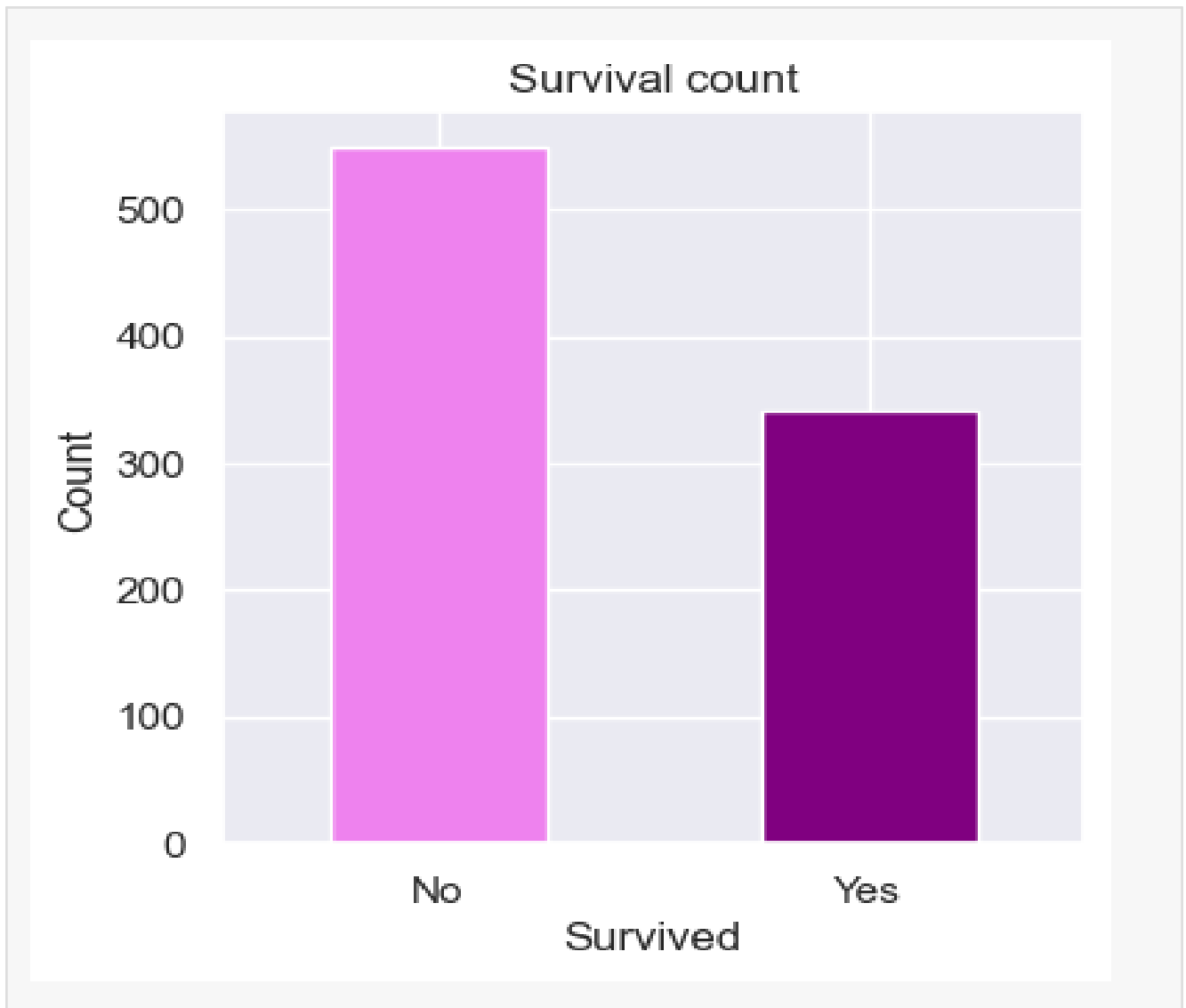
```
print(titanic['survived'].value_counts()) # shows the two values survived and not
```

Output Block

```
survived
0 549
1 342
Name: count, dtype: int64
```

```
plt.figure(figsize=(4,4)) #set the figure size
# Select the specific chart for the figure visualization
titanic['survived'].value_counts().plot(kind='bar', color=['violet','purple'])
plt.title("Survival count") # Give a title
plt.xlabel('Survived') # Define the X-axis label
plt.ylabel('Count') # Define the Y-axis label
plt.xticks([0,1], ['No', 'Yes'], rotation=0) # Binary values replace the labels 0,1
with No and Yes
plt.show()
```

Output Block

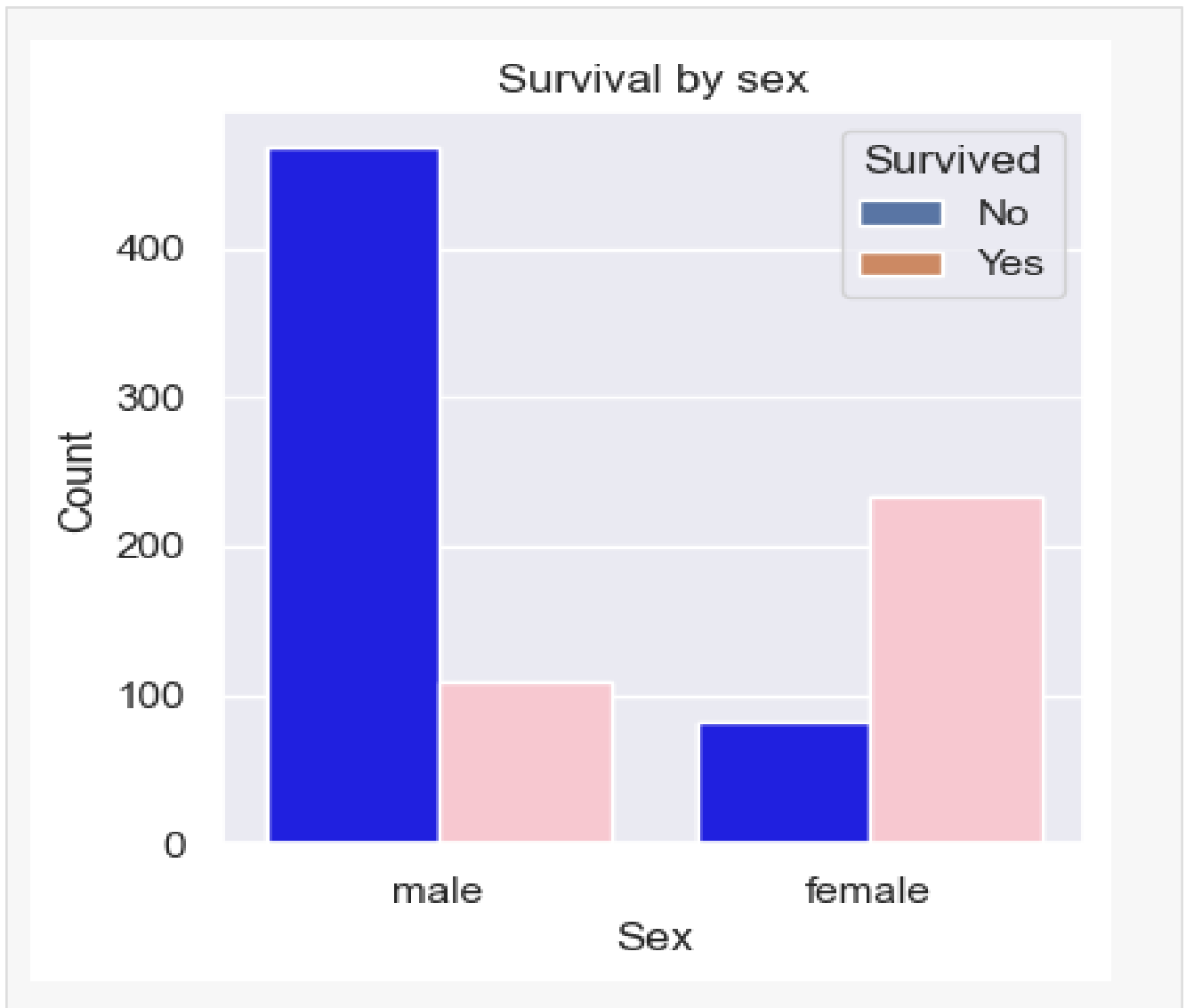


5. Bar Plot: Survival by Sex

Create a bar plot showing survival counts grouped by sex.

```
plt.figure(figsize=(4,4))# Setting the figure size to 4x4
sns.countplot(data=titanic, x="sex", hue="survived") # Will load up default colors
# sns.countplot(data=titanic, x="sex", hue="survived", palette = ['blue', 'pink'])
plt.title("Survival by sex")
plt.xlabel('Sex')
plt.ylabel('Count')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```

Output Block

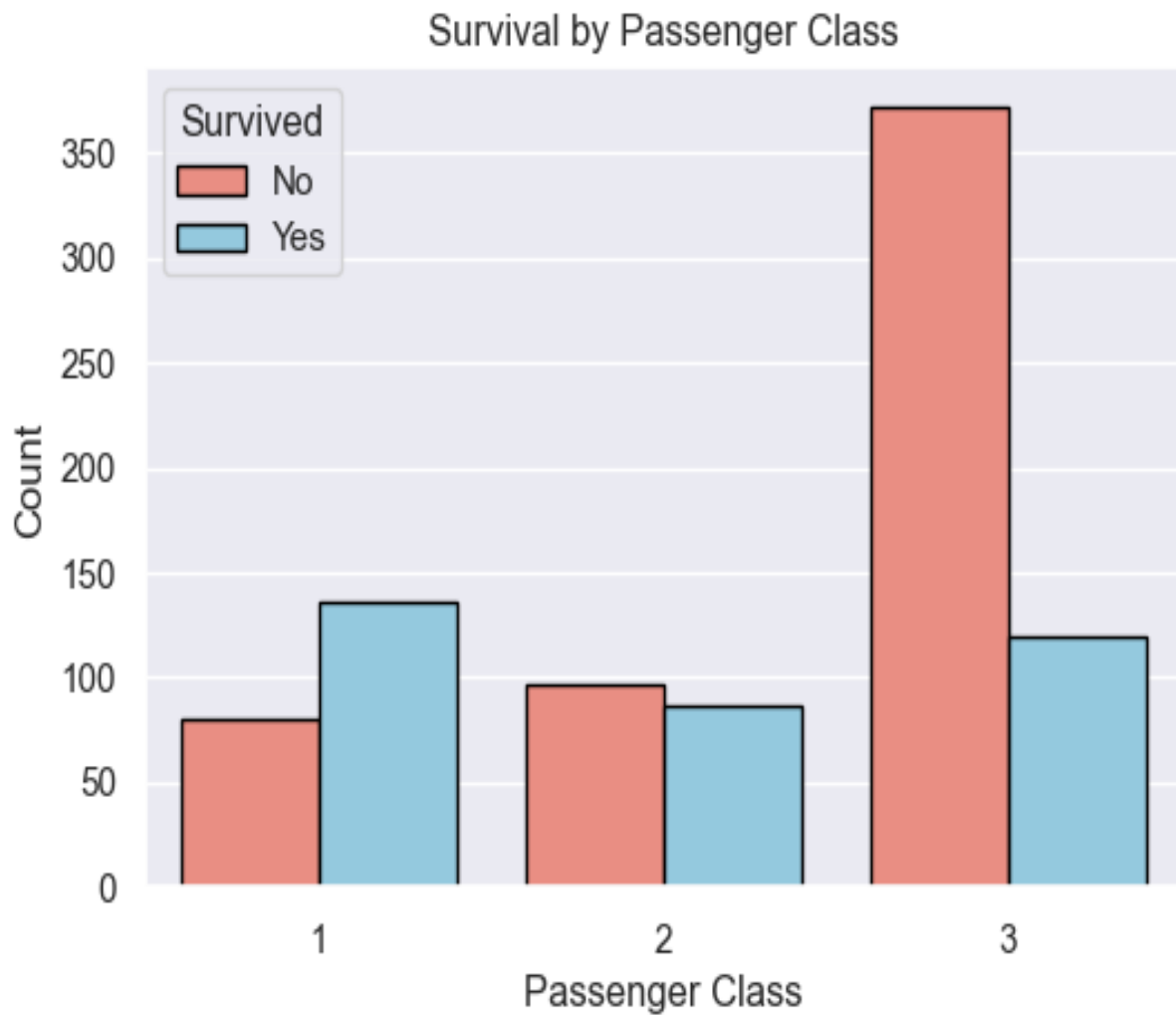


6. Bar Plot: Survival by Passenger Class

Create a bar plot showing survival counts grouped by passenger class.

```
plt.figure(figsize=(6,4))
sns.countplot(data=titanic, x='pclass', hue='survived',edgecolor="black",
palette=['salmon', 'skyblue'])
plt.title('Survival by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Count')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```

Output Block

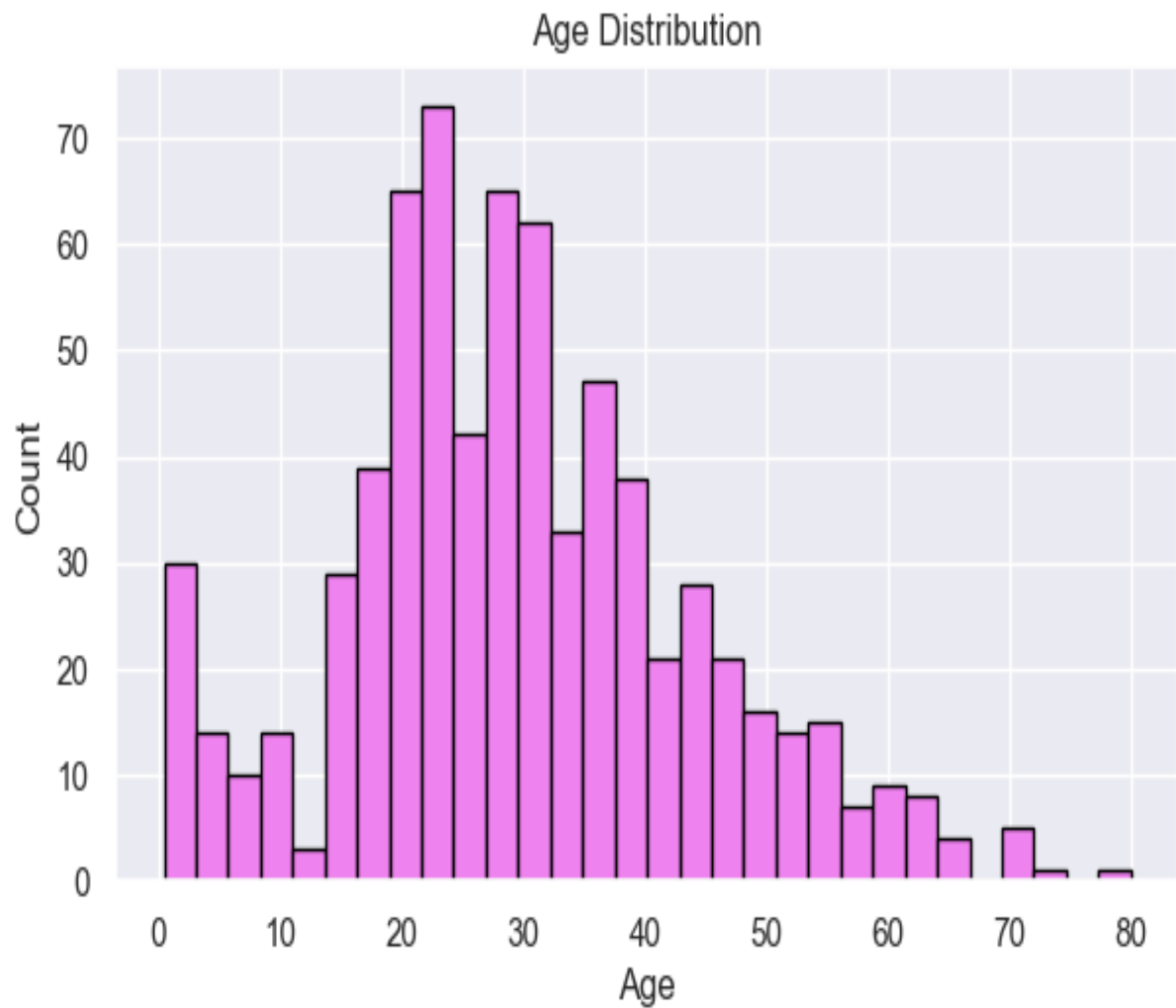


7. Histogram: Age Distribution

Plot a histogram of the age distribution of passengers.

```
plt.figure(figsize=(7,4))
plt.hist(titanic['age'].dropna(), bins=30, color='violet', edgecolor='black')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```

Output Block

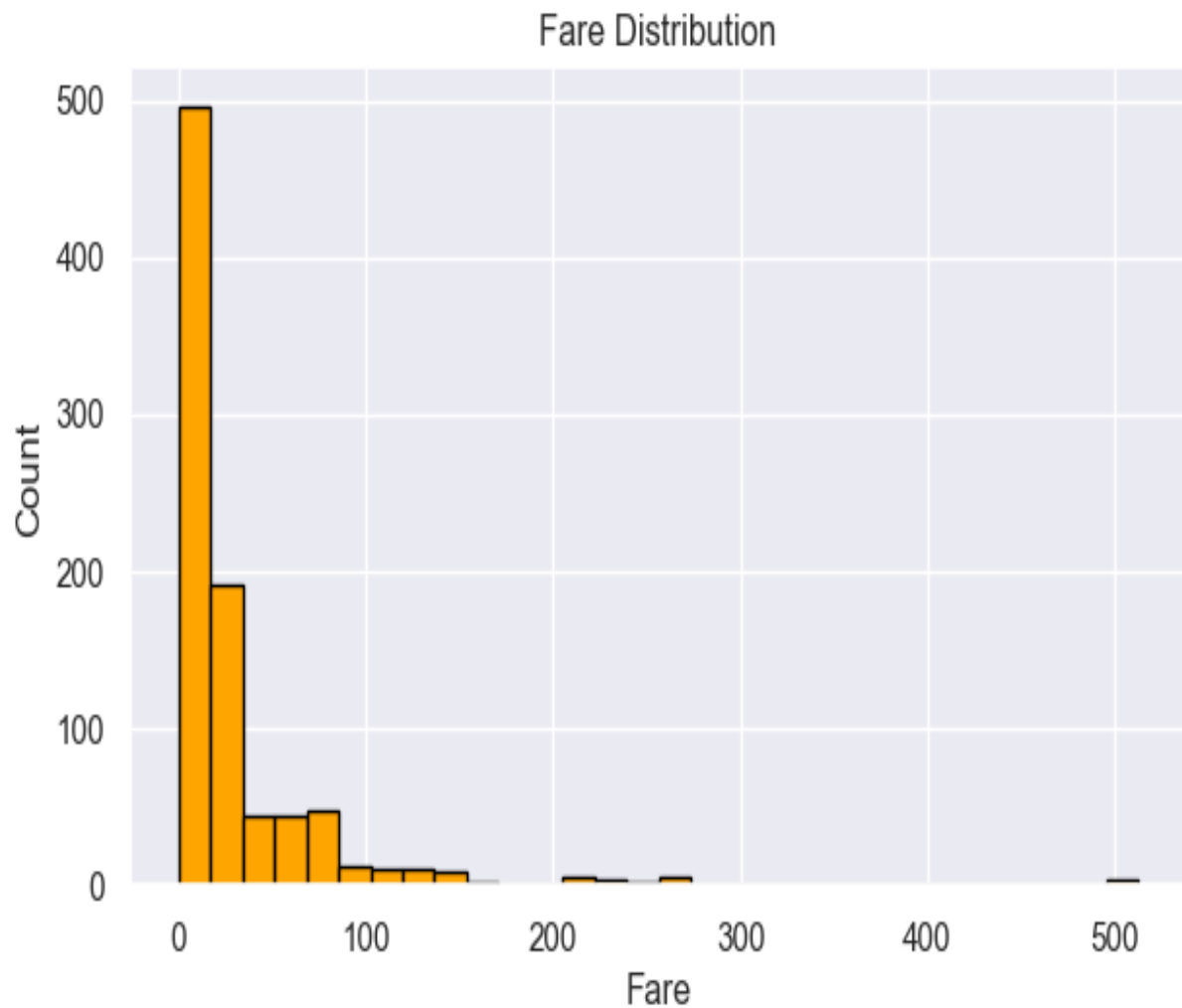


8. Histogram: Fare Distribution

Plot a histogram of the fare distribution.

```
plt.figure(figsize=(7,4))
plt.hist(titanic['fare'].dropna(), bins=30, color='orange', edgecolor='black')
plt.title('Fare Distribution')
plt.xlabel('Fare')
plt.ylabel('Count')
plt.show()
```

Output Block

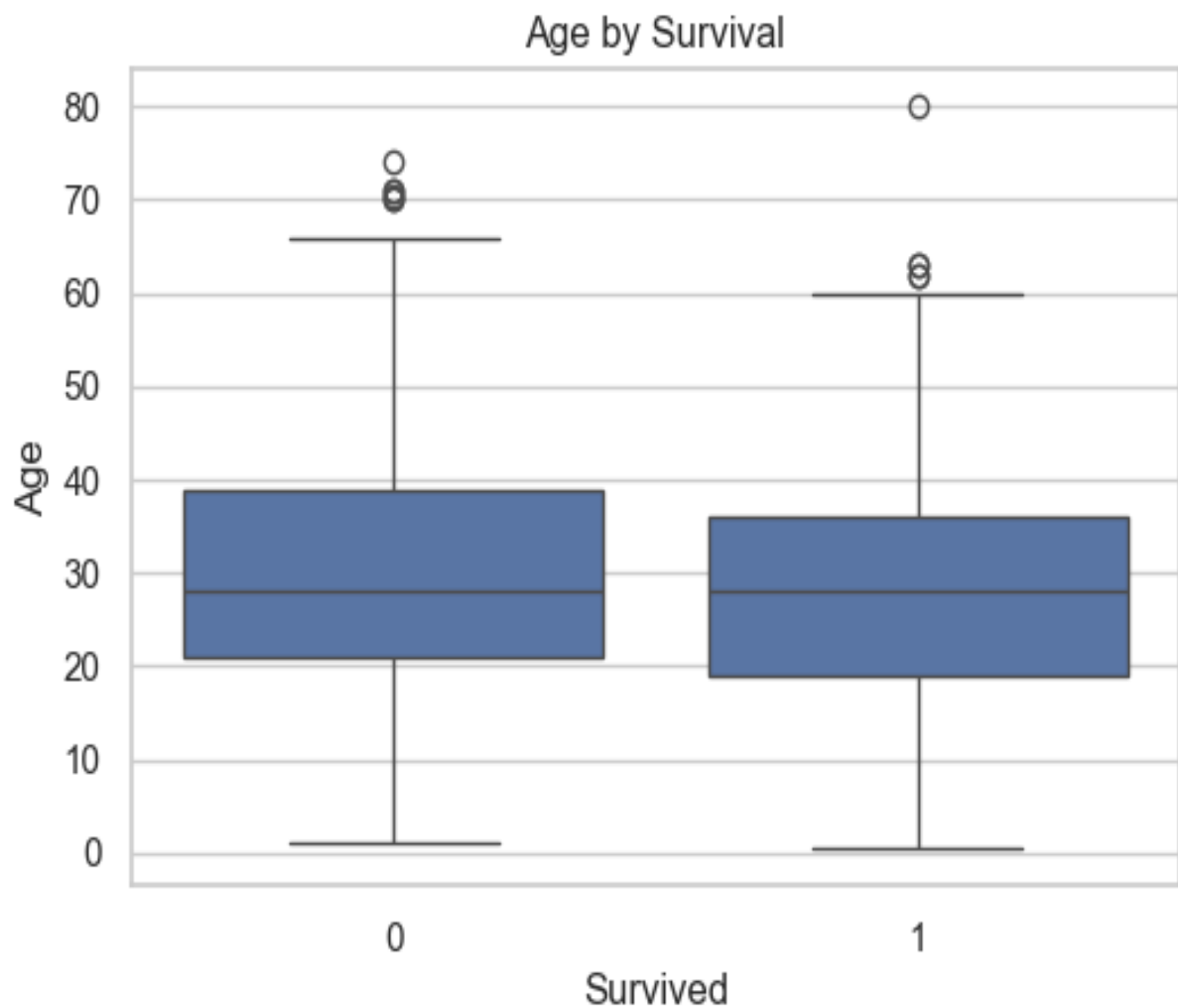


9. Box Plot: Age by Survival

Simple box plot of age grouped by survival status.

```
plt.figure(figsize=(6,4))
sns.boxplot(x='survived', y='age', data=titanic)
plt.title('Age by Survival')
plt.xlabel('Survived')
plt.ylabel('Age')
plt.show()
```

Output Block

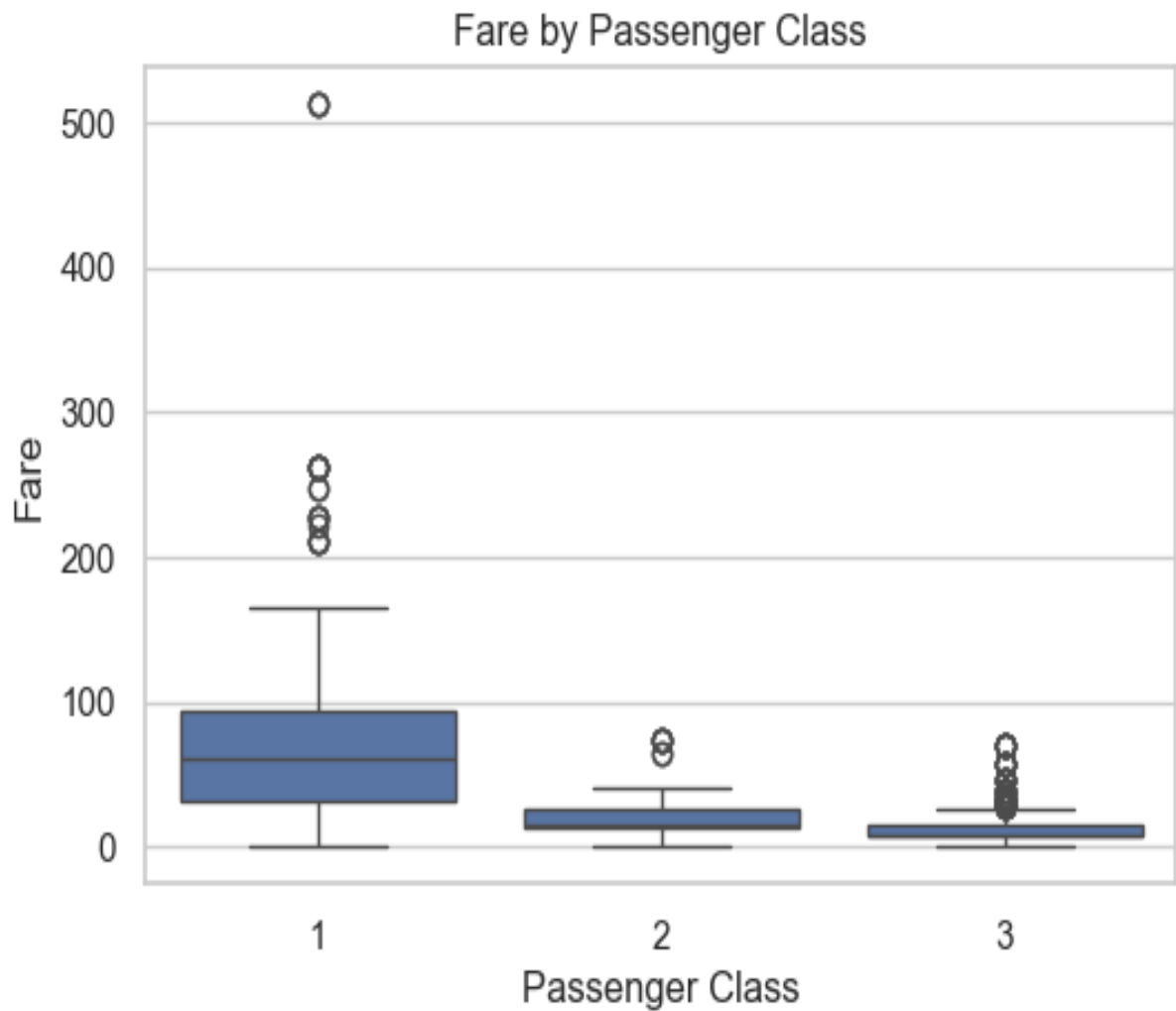


10. Box Plot: Fare by Passenger Class

Simple box plot of fare grouped by passenger class.

```
plt.figure(figsize=(6,4))
sns.boxplot(x='pclass', y='fare', data=titanic)
plt.title('Fare by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Fare')
plt.show()
```

Output Block

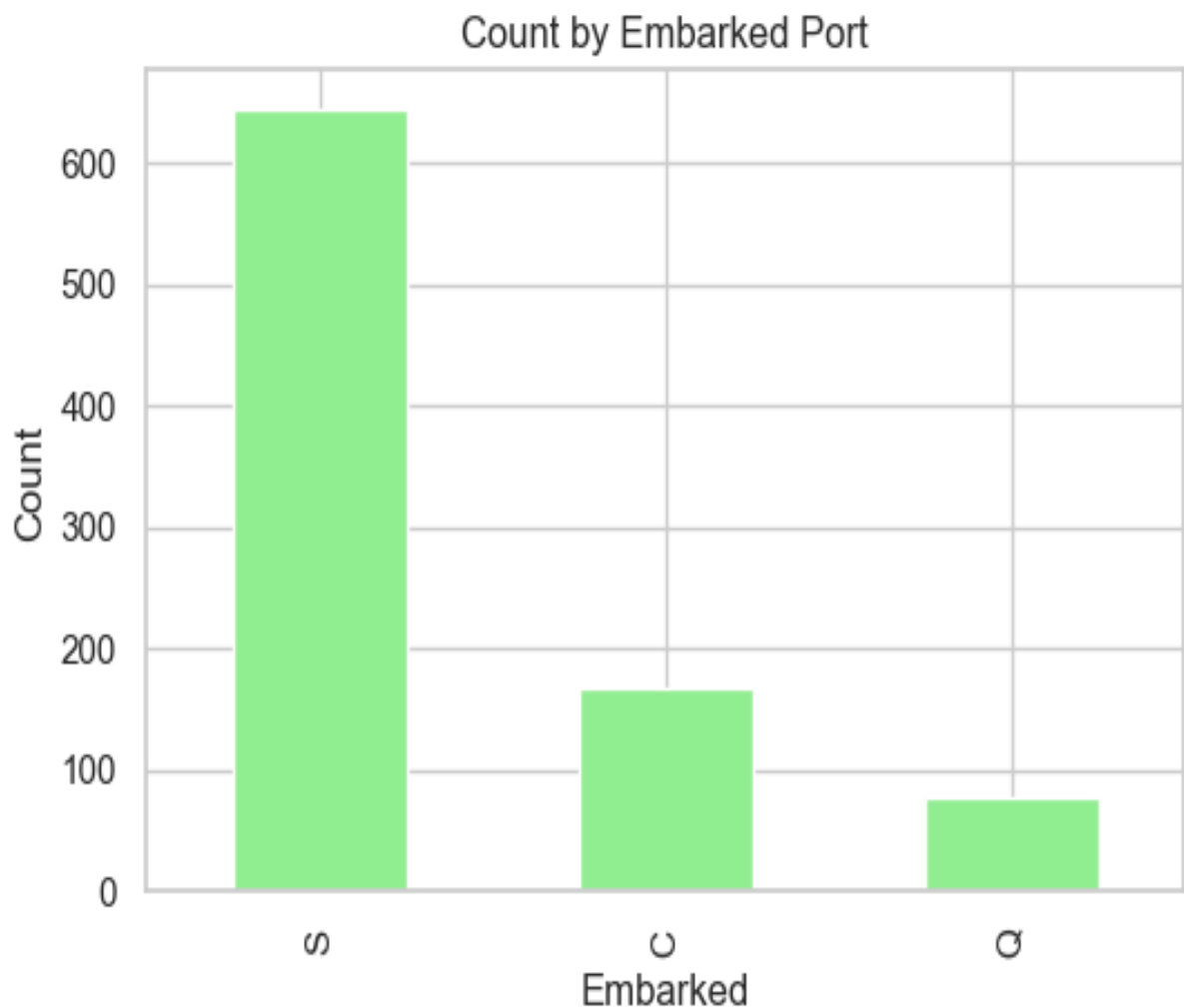


11. Bar Plot: Count by Embarked Port

Simple bar plot of passengers by port of embarkation.

```
plt.figure(figsize=(6,4))
titanic['embarked'].value_counts().plot(kind='bar', color='lightgreen')
plt.title('Count by Embarked Port')
plt.xlabel('Embarked')
plt.ylabel('Count')
plt.show()
```

Output Block

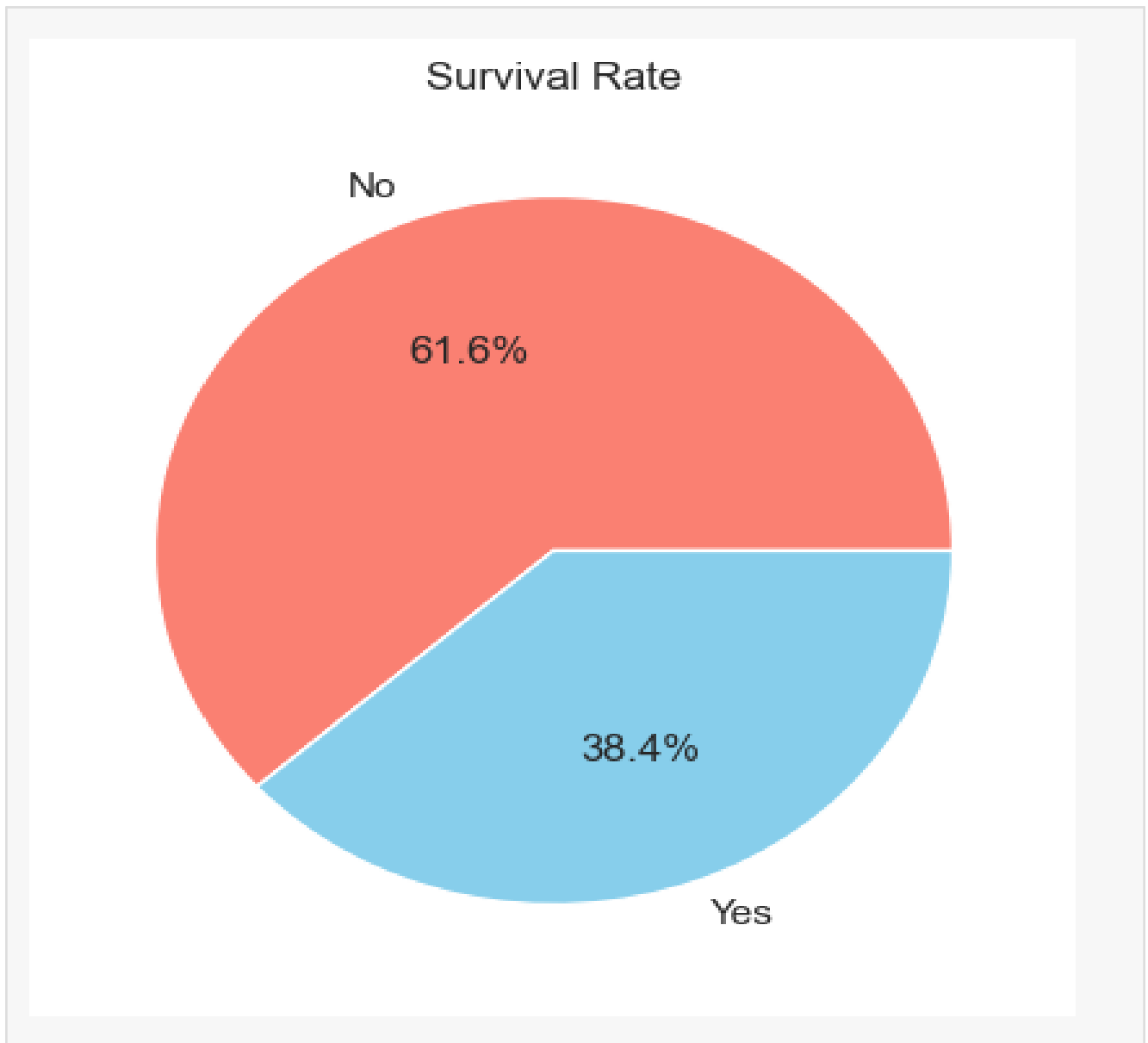


12. Pie Chart: Survival Rate

Simple pie chart showing the proportion of survivors and non-survivors.

```
plt.figure(figsize=(5,5))
titanic['survived'].value_counts().plot.pie(labels=['No', 'Yes'], autopct='%1.1f%%',
colors=['salmon', 'skyblue'])
plt.title('Survival Rate')
plt.ylabel('')
plt.show()
```

Output Block



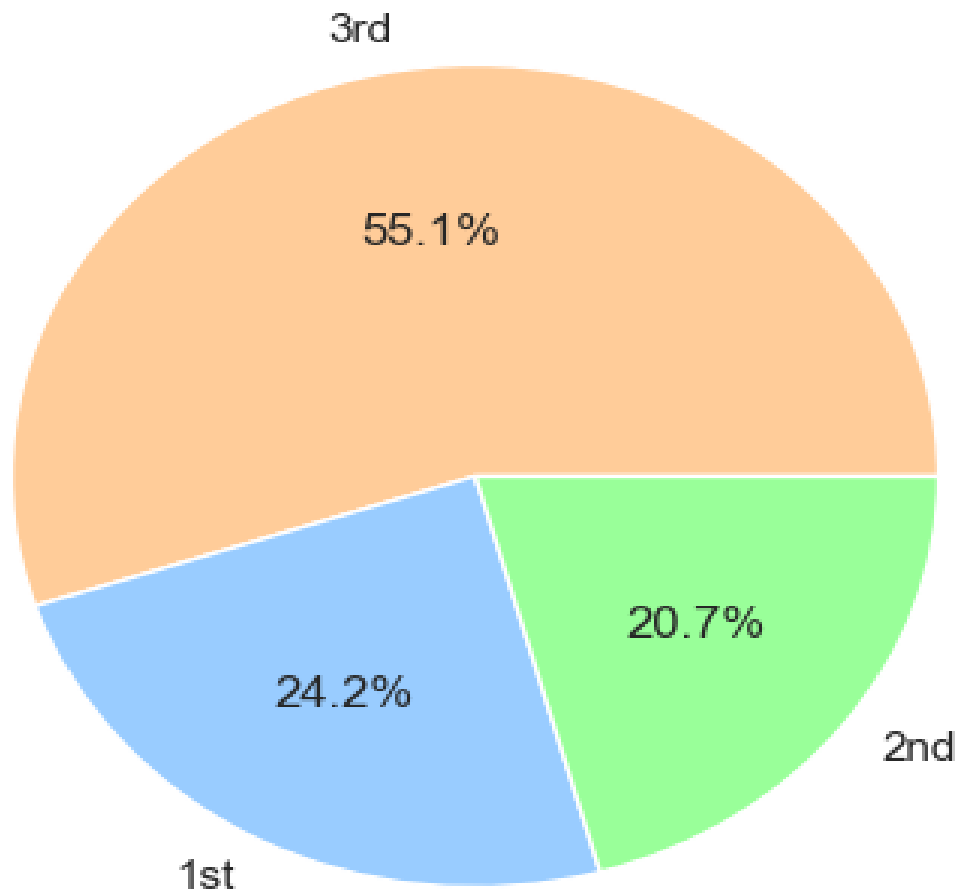
13. Pie Chart: Passenger Class Distribution

Pie chart showing the distribution of passenger classes.

```
plt.figure(figsize=(5,5))
titanic['pclass'].value_counts().plot.pie(labels=['3rd', '1st', '2nd'],
autopct='%1.1f%%', colors=['#ffcc99', '#99ccff', '#99ff99'])
plt.title('Passenger Class Distribution')
plt.ylabel('')
plt.show()
```

Output Block

Passenger Class Distribution



14. Scatter Plot: Age vs Fare

Simple scatter plot of age versus fare.

```
# Create a scatter plot of fare vs age
plt.figure(figsize=(10, 6))
sns.scatterplot(data=titanic, x='age', y='fare')

# Adding titles and labels
plt.title('Titanic: Fare vs Age Scatter Plot')
plt.xlabel('Age')
plt.ylabel('Fare')

# Display the plot
plt.show()
```

Output Block

Titanic: Fare vs Age Scatter Plot

